

HW 6 Contrastive learning

This assignment aims to have you learn how to modify loss functions and simultaneously process multiple feature vectors in both face verification and face recognition tasks by using the CelebA dataset as an example.

CelebFaces Attributes Dataset (CelebA) is a large-scale face attributes dataset with more than 200K celebrity images, each with 40 attribute annotations, covering a wide range of pose variations and background clutter. The dataset is pruned so that the training time is appropriate for the assignment.

This homework is divided into three parts:

1. Face verification with contrastive loss
2. Face verification with triplet loss
3. Face verification with InfoNCE loss
4. Face verification evaluation

Import main libraries

```
In [1]: import pandas as pd
import matplotlib.pyplot as plt
import numpy as np
import os
import os.path as osp
from collections import defaultdict
from PIL import Image
import math
from tqdm.notebook import tqdm
import random

import torch
from torch import nn
import torch.nn.functional as F
from torch import optim
import torchvision
from torch.utils.data import Dataset, DataLoader
from torchvision import transforms
```

```
In [2]: !pwd
```

'pwd' is not recognized as an internal or external command,
operable program or batch file.

Download and unzip the dataset

In [3]: `!pip install gdown`

```
Requirement already satisfied: gdown in c:\users\faith\miniconda3\envs\pattern\lib\site-packages (5.2.1)
Requirement already satisfied: beautifulsoup4 in c:\users\faith\miniconda3\envs\pattern\lib\site-packages (from gdown) (4.14.3)
Requirement already satisfied: filelock in c:\users\faith\miniconda3\envs\pattern\lib\site-packages (from gdown) (3.20.2)
Requirement already satisfied: requests[socks] in c:\users\faith\miniconda3\envs\pattern\lib\site-packages (from gdown) (2.32.5)
Requirement already satisfied: tqdm in c:\users\faith\miniconda3\envs\pattern\lib\site-packages (from gdown) (4.67.1)
Requirement already satisfied: soupsieve>=1.6.1 in c:\users\faith\miniconda3\envs\pattern\lib\site-packages (from beautifulsoup4->gdown) (2.8.1)
Requirement already satisfied: typing-extensions>=4.0.0 in c:\users\faith\miniconda3\envs\pattern\lib\site-packages (from beautifulsoup4->gdown) (4.15.0)
Requirement already satisfied: charset_normalizer<4,>=2 in c:\users\faith\miniconda3\envs\pattern\lib\site-packages (from requests[socks]->gdown) (3.4.4)
Requirement already satisfied: idna<4,>=2.5 in c:\users\faith\miniconda3\envs\pattern\lib\site-packages (from requests[socks]->gdown) (3.11)
Requirement already satisfied: urllib3<3,>=1.21.1 in c:\users\faith\miniconda3\envs\pattern\lib\site-packages (from requests[socks]->gdown) (2.6.2)
Requirement already satisfied: certifi>=2017.4.17 in c:\users\faith\miniconda3\envs\pattern\lib\site-packages (from requests[socks]->gdown) (2025.11.12)
Requirement already satisfied: PySocks!=1.5.7,>=1.5.6 in c:\users\faith\miniconda3\envs\pattern\lib\site-packages (from requests[socks]->gdown) (1.7.1)
Requirement already satisfied: colorama in c:\users\faith\miniconda3\envs\pattern\lib\site-packages (from tqdm->gdown) (0.4.6)
```

In []: `!gdown --id 1_2oATmA0Jw61qs7I7kAG9Q2LHNZSQX7A`

In []: `# !unzip /content/Large_prepared_data.zip`

```
In [6]: import os, zipfile

zip_path = 'large_prepared_data.zip'
extract_to = '.'

os.makedirs(extract_to, exist_ok=True)
with zipfile.ZipFile(zip_path, 'r') as zf:
    zf.extractall(extract_to)

print('Done:', extract_to)
```

Done: .

Common dataset

```
In [7]: class FaceDataset(Dataset):
        def __init__(self, root_dir, transform=None):
```

```

self.root_dir = root_dir
self.transform = transform
self.label_df = pd.read_csv(f'{self.root_dir}/label_df.csv')

def __len__(self):
    return len(self.label_df)

def __getitem__(self, idx):
    data = self.label_df.iloc[idx]
    img = Image.open(f"{self.root_dir}/{data['filename']}").convert('RGB')
    if self.transform is not None:
        transformed_img = self.transform(img)
    else:
        transformed_img = None
    identity = data['identity']
    return transformed_img, identity, np.array(img)

```

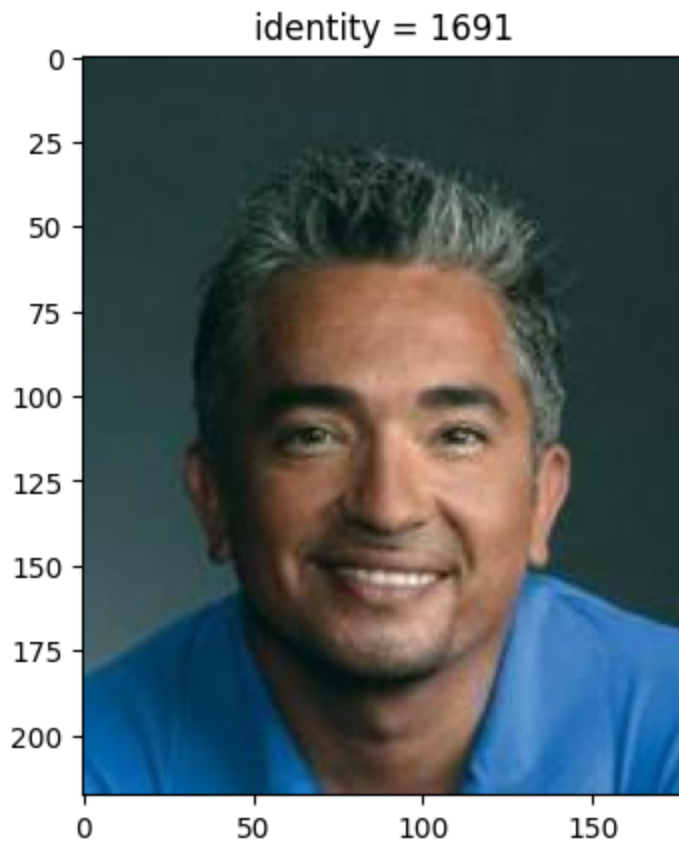
This is an example to display a face image with its identity

```

In [8]: import matplotlib.pyplot as plt
face_dataset = FaceDataset(root_dir='large_prepared_data/test')

for idx in range(4):
    _, identity, img = face_dataset[idx]
    plt.title(f'identity = {identity}')
    plt.imshow(img)
    plt.show()

```







Part 1: Face verification with contrastive loss

The objective of the face verification task is to validate whether the face image x has the identity y by comparing it to the face database of the claimed identity. If the face similarity score between x and the face in the database of y is above a certain threshold, the image is then verified; otherwise, the identity is rejected.

In HW3, you have learned to calculate a similarity score based on compact image representation using a PCA / Fisher projection. Therefore, in this part, you will instead implement a more contemporary method by training the NN to propose a compact representation (feature vector) by using a Siamese network and contrastive loss (Chopra et al., 2005, <https://ieeexplore.ieee.org/stamp/stamp.jsp?tp=&arnumber=1467314>).

In contrast to the classification task that forces the model to learn all possible classes of objects, the general idea of object verification is based on contrastive learning, a framework that teaches the model to distinguish the two objects from each other. For a contrastive loss, the model will receive a pair of image and then learn to recognize whether both of them has the same identity by encouraging the feature vector of the same identity to come closer and different one to move away from each other.

1.1 (TODO) Dataset and DataLoader

First, we will start by writing a dataloader. The contrastive loss learns whether a pair of images have the same identity. Therefore, in this subsection, you have to write a dataloader that returns the tuple `(img1, img2, is_same_identity (bool))`.

File structure of this dataset

```
- large_prepared_data/
  └─ train/
    |   - {img1}.jpg
    |   - {img2}.jpg
    |   - label_df.csv
    └─ val/
      |   - {img3}.jpg
      |   - {img4}.jpg
      |   - label_df.csv
      └─ test/
        |   - {img5}.jpg
        |   - {img6}.jpg
        |   - label_df.csv
```

Note

label_df.csv has 2 columns that are the filename such as 001257.jpg and the identity such as 1691

Instructions

TODO 1-4: Fill in the missing code in the cells below.

TODO 1-2: Organize the dataset for simple data access.

TODO 3: Randomly select the datapoint from the dataset and format it to be ready for training.

TODO 4: Initialize the dataloader.

```
In [9]: class SiameseDataset(Dataset):
        def __init__(self, root_dir, transform=None):
            """
            The dataset of siamese network
            [Args]
            - root_dir = root directory of the dataset
            - transform = transformations for images
            """
            np.random.seed(123)
            random.seed(123)
            self.root_dir = root_dir
            self.transform = transform
            label_df = pd.read_csv(f'{root_dir}/label_df.csv')
            self.num_images = len(label_df)
```

```

label_df = label_df.groupby('identity')['filename'].apply(list).reset_index()
self.load_images_to_memory_(label_df)

def load_images_to_memory_(self, label_df):
    """
        Load all images into memory
        [Args]
        - label_df = The dataframe containing the identities and the filenames of i
    """
    # TODO 1: Load images to `self.data` according to the below structure
    # and `self.images`, `self.identities` following by idx
    # Note: identity{i}: str, image{i}: PIL.Image
    # (convert them to RGB as well)
    # e.g. self.data = {
    #     'identity1': [image1, image2],
    #     'identity2': [image3, image4, image5],
    # }
    #
    # identity{i}: str, image{i}: PIL.Image

    # self.data is just a dict. key: string, value: List of Image
    # self.images and self.identities is just a flat list of the map
    self.identities = []
    self.images = []
    self.data = {}
    for idx, row in label_df.iterrows():
        # row is pd.Series with column name as index
        identity = row['identity']
        filenames = row['filenames']

        tmp_imgs = [Image.open(f"{self.root_dir}/{filename}").convert("RGB") for filename in filenames]
        self.images += tmp_imgs
        self.identities += [identity] * len(filenames)
        self.data[identity] = tmp_imgs

    # TODO 2: keep all unique identities as list with `self.unique_identities`
    # in `self.unique_identities` as a numpy array.
    self.unique_identities = label_df["identity"].unique() # label_df is already
    # self.unique_identities = label_df["identity"].values
    # self.unique_identities = np.array(label_df["identity"].tolist())

def __len__(self):
    return self.num_images

def __getitem__(self, idx):
    """
        Return a pair of image together with its label
        [Args]
        - idx: int
        [Return]
        - img1: torch.FloatTensor
        - img2: torch.FloatTensor
        - label: torch.FloatTensor = 1 (same class), 0 (different class)
    """
    # TODO 3: randomly sample a pair of images
    # Note: idx is even, it should return the same class pair and otherwise

```

```

# Please use label = 1 for the same class pair
# and label = 0 for the different class pair
img1 = self.images[idx]
identity = self.identities[idx]

if (idx % 2 == 0):
    label = 1 # Same class
    img2_idx = np.random.randint(0, len(self.data[identity]))
    img2 = self.data[identity][img2_idx]
    # Can uncomment while loop out if I do not allow img2 to be the same picture
    # while ((len(self.data[identity]) != 1) and (img2 is img1)):
    #     img2_idx = np.random.randint(0, len(self.data[identity]))
    #     self.data[identity][img2_idx]

else:
    label = 0 # Different class
    random_identity = np.random.choice(self.unique_identities)
    while (random_identity == identity):
        random_identity = np.random.choice(self.unique_identities)
    img2_idx = np.random.randint(0, len(self.data[random_identity]))
    img2 = self.data[random_identity][img2_idx]

# transform to tensor
if (self.transform):
    img1 = self.transform(img1)
    img2 = self.transform(img2)

return img1, img2, torch.from_numpy(np.array([label], dtype=np.float32))

```

```

In [10]: img_size = 224
train_transform = transforms.Compose([
    transforms.Resize((256, 256)),
    transforms.CenterCrop(img_size),
    transforms.RandomHorizontalFlip(p=0.5),
    transforms.ToTensor(),
    transforms.Normalize(mean=[0.5319, 0.4399, 0.3929],
                        std=[0.3076, 0.2898, 0.2907])
])

# val_transform : no augmentation
val_transform = transforms.Compose([
    transforms.Resize((256, 256)),
    transforms.CenterCrop(img_size),
    transforms.ToTensor(),
    transforms.Normalize(mean=[0.5319, 0.4399, 0.3929],
                        std=[0.3076, 0.2898, 0.2907])
])

train_batch_size = 16
val_batch_size = 16
test_batch_size = 16

# TODO 4: declare the datasets and the dataloaders
train_siamese_dataset = SiameseDataset(root_dir="large_prepared_data/train", transf
train_siamese_dataloader = DataLoader(train_siamese_dataset, batch_size=train_batch

```

```
val_siamese_dataset = SiameseDataset(root_dir="large_prepared_data/val", transform=  
val_siamese_dataloader = DataLoader(val_siamese_dataset, batch_size=val_batch_size,  
  
test_siamese_dataset = SiameseDataset(root_dir="large_prepared_data/test", transform=  
test_siamese_dataloader = DataLoader(test_siamese_dataset, batch_size=test_batch_si
```

1.2 (TODO) Siamese network

After the dataloader is initialized, we then build a siamese network. Section 1.5 will explain how a siamese network works in full detail.

Siamese network is a typical CNN that consists of three modules:

1. A feature extractor (ResNet18) for extracting the feature map from an image.
2. A global pooling for reducing the image dimension.
3. A fully connected layer for compressing the feature vector

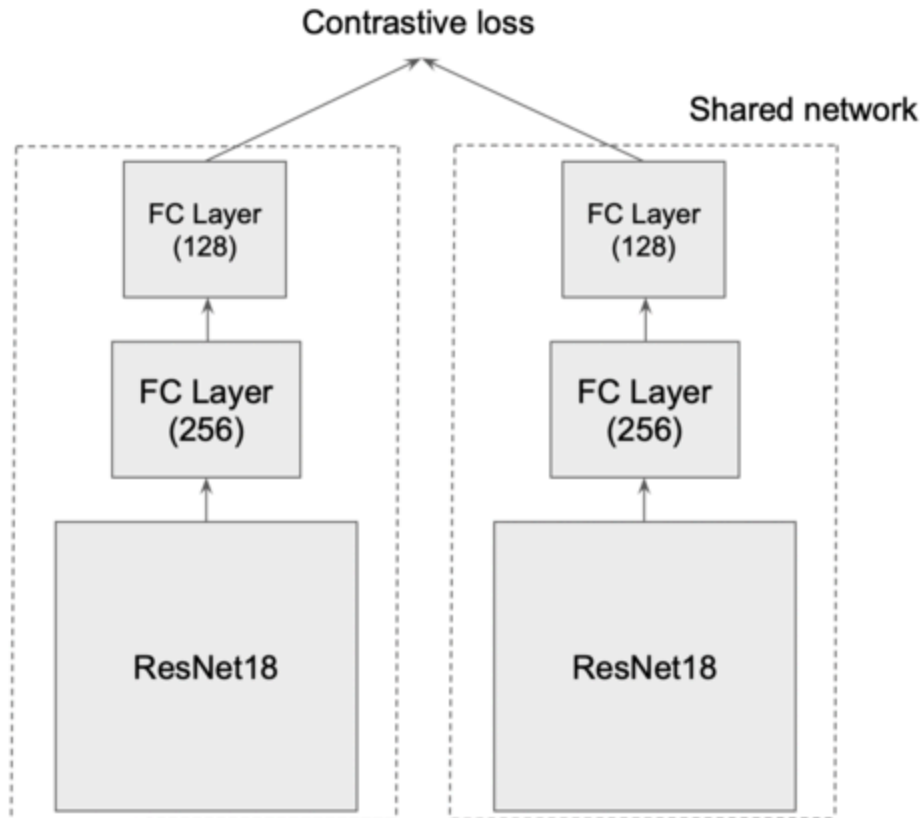
Every fully connected layer is followed by ReLU activations.

TODO 5: Implement a siamese network based on the description.

Note

1. You can use the ResNet18 from the `torchvision` library (How to use torchvision: <https://pytorch.org/vision/stable/models.html>)
2. We will not use pretrained weights.

Figure 1 Siamese network



```
In [11]: from torchvision.models import resnet18
# we will not use the pre-trained weight just use its architecture

class SiameseNetwork(nn.Module):
    # TODO 5: implement the siamese network
    def __init__(self):
        super().__init__()
        self.resnet = resnet18(weights=None)
        self.resnet.fc = nn.Identity() # resnet is also nn.Module which is able to
        # check structure of thhe model by print(self.resnet)
        self.fc = nn.Sequential(
            nn.Linear(512, 256),
            nn.ReLU(),
            nn.Linear(256, 128),
            nn.ReLU(),
        )

    def extract_feature(self, x):
        feature = self.resnet(x)
        output = self.fc(feature.flatten(start_dim=1))
        return output

    def forward(self, input1, input2):
        output1 = self.extract_feature(input1)
        output2 = self.extract_feature(input2)
        return output1, output2
        # We're going to move this vector (128)
```

1.3 (TODO) Contrastive loss

A contrastive loss is a loss used to minimize the dissimilarity between two images by encouraging the feature vector of the same identity to come closer and different one further than a constant margin m to move away from each other.

The contrastive loss is mathematically defined as:

$$L(\text{contrastive loss}) = \begin{cases} d(r_1, r_2) & \text{if } \text{identity}(r_1) = \text{identity}(r_2) \\ \max(0, m - d(r_1, r_2)) & \text{if } \text{identity}(r_1) \neq \text{identity}(r_2) \end{cases}$$

where

- $d(r_1, r_2)$ = euclidean distance between r_1 and r_2
- m = margin
- $\text{identity}(x)$ = the identity of x
- r_1 = the feature vector of the first image
- r_2 = the feature vector of the second image

The term $d(r_1, r_2)$ is the distance between the two feature vectors. The contrastive loss minimizes the distance between the feature vectors of the same identity (positive pair) but maximizes the distance of the different identities (negative pair).

The margin m is used to prevent the loss from collapsing to a trivial solution. For instance, when $m = 0$, the model could achieve $L = 0$ just by exploiting the objective by setting $r_1 = r_2$, and the model, as a result, would learn nothing useful.

HINT

Many torch functions often have the same functionality as NumPy functions, even sharing the same function name. Therefore, if you are struggling with this part, you might write the loss using NumPy first and then convert it to the torch function (<https://pytorch.org/docs/stable/torch.html>) later. **Avoid using "if statements" to make the training faster. You have already learned many tricks that convert if statements into a single equation.**

TODO 6: Implement a contrastive loss based on the description above.

```
In [12]: import torch
from torch import nn
import torch.nn.functional as F

class ContrastiveLoss(torch.nn.Module):
    # TODO 6: implement the contrastive loss
    def __init__(self, margin):
        super().__init__()
        self.margin = margin
```

```
def forward(self, output1, output2, label):
    dist = (output1 - output2) ** 2 # element-wise
    dist = torch.sum(dist, dim=1) # maintain the batch dimension
    dist = torch.sqrt(dist + 1e-8) # in case root 0 gets 0 and its derivative will
    label = label.squeeze() # remove all dimension of size 1 because it usually com
    loss_contrastive = (dist * (label)) + (torch.clamp(self.margin - dist, min=0.0))
    return loss_contrastive.mean() # mean for batch size
```

1.4 Initiazing the model, criterion, optimizer and scheduler

```
In [13]: siamese_margin = 2
         learning_rate = 1e-4

         siamese_model = SiameseNetwork()
         siamese_criterion = ContrastiveLoss(margin=siamese_margin)
         siamese_optimizer = optim.Adam(siamese_model.parameters(), lr=learning_rate)
         siamese_scheduler = optim.lr_scheduler.ReduceLROnPlateau(siamese_optimizer, 'min',
```

1.5 (TODO) Training loop

The training procedure of a siamese network consists of the following steps:

1. Forward pass the `img1` .
2. Forward pass the `img2` .
3. Calculate the distance between the feature vector of `img1` and `img2` ($d(r_1, r_2)$).
4. Use the distance in step 3 as a loss and update the model.
5. Repeat step 1-4 until satisfied.

As you would notice, both first and second step shares the same network weights. Therefore, the word "Siamese" in the siamese network originates from the "Siamese twins" since the network performs two forward passes to compare whether the feature vectors have the same identity by using the same set of network parameters (shared parameters).

TODO 7-8: Feed pairs of images to the network, compute contrastive loss to measure the dissimilarity of pairs of face images and update the network.

TODO 9: Feed a pair of images from validation set to the network and compute the validation loss.

Trivia

The Siamese twin is a conjoined twin brother born in Siam who later move to the US (<https://th.wikipedia.org/wiki/%E0%B8%AD%E0%B8%B4%E0%B8%99-%E0%B8%88%E0%B8%B1%E0%B8%99>).


```

In [14]: num_epochs = 20
device = "cuda" if torch.cuda.is_available() else "cpu"
siamese_model.to(device)

os.makedirs('weights', exist_ok=True)
best_weights_path = 'weights/best_siamese_weights.pth'

train_losses = []
val_losses = []
min_val_loss = float('inf')

for epoch in tqdm(range(num_epochs)):
    siamese_model.train()
    total_train_loss = 0

    for img1, img2, label in tqdm(train_siamese_dataloader):

        # TODO 7: feed data to model and compute Loss
        img1, img2, label = img1.to(device), img2.to(device), label.to(device)

        siamese_optimizer.zero_grad()

        output1, output2 = siamese_model(img1, img2)

        train_loss = siamese_criterion(output1, output2, label)

        # TODO 8: back propagate
        train_loss.backward()
        siamese_optimizer.step()

        total_train_loss += train_loss.item()

    current_train_loss = total_train_loss / len(train_siamese_dataloader)
    train_losses.append(current_train_loss)

    total_val_loss = 0
    siamese_model.eval()
    for val_img1, val_img2, val_label in val_siamese_dataloader:
        # TODO 9: feed data to model and compute Loss
        with torch.no_grad():
            val_img1, val_img2, val_label = val_img1.to(device), val_img2.to(device)

            output1, output2 = siamese_model(val_img1, val_img2)

            val_loss = siamese_criterion(output1, output2, val_label)

            total_val_loss += val_loss.item()
    current_val_loss = total_val_loss / len(val_siamese_dataloader)
    val_losses.append(current_val_loss)
    if current_val_loss < min_val_loss:
        min_val_loss = current_val_loss
        torch.save(siamese_model.state_dict(), best_weights_path)
    print(f'Epoch {epoch+1} - Train loss = {current_train_loss:.4f} - Val loss = {current_val_loss:.4f}')
    siamese_scheduler.step(current_val_loss)

```

```

0%|          | 0/20 [00:00<?, ?it/s]
0%|          | 0/153 [00:00<?, ?it/s]
Epoch 1 - Train loss = 0.8928 - Val loss = 0.7963 - best min_val_loss = 0.7963 - lr
= 0.00010000
0%|          | 0/153 [00:00<?, ?it/s]
Epoch 2 - Train loss = 0.8010 - Val loss = 0.7431 - best min_val_loss = 0.7431 - lr
= 0.00010000
0%|          | 0/153 [00:00<?, ?it/s]
Epoch 3 - Train loss = 0.8082 - Val loss = 0.7986 - best min_val_loss = 0.7431 - lr
= 0.00010000
0%|          | 0/153 [00:00<?, ?it/s]
Epoch 4 - Train loss = 0.8052 - Val loss = 0.8011 - best min_val_loss = 0.7431 - lr
= 0.00010000
0%|          | 0/153 [00:00<?, ?it/s]
Epoch 5 - Train loss = 0.8043 - Val loss = 0.8220 - best min_val_loss = 0.7431 - lr
= 0.00010000
0%|          | 0/153 [00:00<?, ?it/s]
Epoch 6 - Train loss = 0.7866 - Val loss = 0.7553 - best min_val_loss = 0.7431 - lr
= 0.00010000
0%|          | 0/153 [00:00<?, ?it/s]
Epoch 7 - Train loss = 0.7695 - Val loss = 0.7372 - best min_val_loss = 0.7372 - lr
= 0.00010000
0%|          | 0/153 [00:00<?, ?it/s]
Epoch 8 - Train loss = 0.7445 - Val loss = 0.6890 - best min_val_loss = 0.6890 - lr
= 0.00010000
0%|          | 0/153 [00:00<?, ?it/s]
Epoch 9 - Train loss = 0.7297 - Val loss = 0.6771 - best min_val_loss = 0.6771 - lr
= 0.00010000
0%|          | 0/153 [00:00<?, ?it/s]
Epoch 10 - Train loss = 0.7453 - Val loss = 0.6883 - best min_val_loss = 0.6771 - lr
= 0.00010000
0%|          | 0/153 [00:00<?, ?it/s]
Epoch 11 - Train loss = 0.7673 - Val loss = 0.7788 - best min_val_loss = 0.6771 - lr
= 0.00010000
0%|          | 0/153 [00:00<?, ?it/s]
Epoch 12 - Train loss = 0.7444 - Val loss = 0.6819 - best min_val_loss = 0.6771 - lr
= 0.00010000
0%|          | 0/153 [00:00<?, ?it/s]
Epoch 13 - Train loss = 0.7666 - Val loss = 0.6397 - best min_val_loss = 0.6397 - lr
= 0.00010000
0%|          | 0/153 [00:00<?, ?it/s]
Epoch 14 - Train loss = 0.7187 - Val loss = 0.6915 - best min_val_loss = 0.6397 - lr
= 0.00010000
0%|          | 0/153 [00:00<?, ?it/s]
Epoch 15 - Train loss = 0.6887 - Val loss = 0.6900 - best min_val_loss = 0.6397 - lr
= 0.00010000
0%|          | 0/153 [00:00<?, ?it/s]
Epoch 16 - Train loss = 0.6931 - Val loss = 0.6091 - best min_val_loss = 0.6091 - lr
= 0.00010000
0%|          | 0/153 [00:00<?, ?it/s]
Epoch 17 - Train loss = 0.6951 - Val loss = 0.6296 - best min_val_loss = 0.6091 - lr
= 0.00010000
0%|          | 0/153 [00:00<?, ?it/s]
Epoch 18 - Train loss = 0.7023 - Val loss = 0.6327 - best min_val_loss = 0.6091 - lr
= 0.00010000
0%|          | 0/153 [00:00<?, ?it/s]

```

Epoch 19 - Train loss = 0.6849 - Val loss = 0.6065 - best min_val_loss = 0.6065 - lr = 0.00010000
 0%| | 0/153 [00:00<?, ?it/s]
 Epoch 20 - Train loss = 0.6589 - Val loss = 0.5573 - best min_val_loss = 0.5573 - lr = 0.00010000

1.6 Visualization

This visualization displays pairs of images together with the distance between those pairs.

```
In [15]: # Showing images
def imshow(img, text=None):
    npimg = img.numpy()
    plt.axis("off")
    if text:
        plt.text(120, 8, text, style='italic', fontweight='bold',
                 bbox={'facecolor': 'white', 'alpha': 0.8, 'pad': 10})

    plt.imshow(np.transpose(npimg, (1, 2, 0)))
    plt.show()

siamese_model.load_state_dict(torch.load(best_weights_path))
siamese_model.eval()

test_img1s, test_img2s, test_labels = next(iter(test_siamese_dataloader))
test_img1s, test_img2s, test_labels = test_img1s.to(device), test_img2s.to(device),
with torch.no_grad():
    test_out1s, test_out2s = siamese_model(test_img1s, test_img2s)

class UnNormalize(object):
    def __init__(self, mean, std):
        self.mean = mean
        self.std = std

    def __call__(self, tensor):
        """
        Args:
            tensor (Tensor): Tensor image of size (C, H, W) to be normalized.
        Returns:
            Tensor: Normalized image.
        """
        for t, m, s in zip(tensor, self.mean, self.std):
            t.mul_(s).add_(m) # px * std + mean
            # The normalize code -> t.sub_(m).div_(s)
        return tensor

unnormalizer = UnNormalize(mean=[0.5319, 0.4399, 0.3929], # for RGB
                           std=[0.3076, 0.2898, 0.2907])
# need to normalize because nn takes 0-1 not 0-255
for test_out1, test_out2, test_img1, test_img2, test_label in zip(test_out1s, test_out2s,
                           test_img1s, test_img2s, test_labels):
    test_img1 = unnormalizer(test_img1.detach().cpu().unsqueeze(0))
    test_img2 = unnormalizer(test_img2.detach().cpu().unsqueeze(0))
    concatenated = torch.cat((test_img1, test_img2), 0)
```

```
distance = F.pairwise_distance(test_out1.unsqueeze(0), test_out2.unsqueeze(0))  
imshow(torchvision.utils.make_grid(concatenated), f'Label = {int(test_label[0])}
```

Clipping input data to the valid range for imshow with RGB data ([0..1] for floats or [0..255] for integers). Got range [0.0..1.1659949].



Clipping input data to the valid range for imshow with RGB data ([0..1] for floats or [0..255] for integers). Got range [0.0..1.1742941].



Clipping input data to the valid range for imshow with RGB data ([0..1] for floats or [0..255] for integers). Got range [0.0..1.1742941].



Clipping input data to the valid range for imshow with RGB data ([0..1] for floats or [0..255] for integers). Got range [0.0..1.1742941].



Clipping input data to the valid range for imshow with RGB data ([0..1] for floats or [0..255] for integers). Got range [0.0..1.1120508].



Clipping input data to the valid range for imshow with RGB data ([0..1] for floats or [0..255] for integers). Got range [0.0..1.1742941].



Clipping input data to the valid range for imshow with RGB data ([0..1] for floats or [0..255] for integers). Got range [0.0..1.1162004].



Clipping input data to the valid range for imshow with RGB data ([0..1] for floats or [0..255] for integers). Got range [0.0..1.1742941].



Clipping input data to the valid range for imshow with RGB data ([0..1] for floats or [0..255] for integers). Got range [0.0..1.1203499].



Clipping input data to the valid range for imshow with RGB data ([0..1] for floats or [0..255] for integers). Got range [0.0..1.1742941].



Clipping input data to the valid range for imshow with RGB data ([0..1] for floats or [0..255] for integers). Got range [0.0..1.1742941].



Clipping input data to the valid range for imshow with RGB data ([0..1] for floats or [0..255] for integers). Got range [0.0..1.1742941].



Clipping input data to the valid range for imshow with RGB data ([0..1] for floats or [0..255] for integers). Got range [0.0..1.1369481].



Clipping input data to the valid range for imshow with RGB data ([0..1] for floats or [0..255] for integers). Got range [0.0..1.1618454].



Clipping input data to the valid range for imshow with RGB data ([0..1] for floats or [0..255] for integers). Got range [0.0..1.1742941].

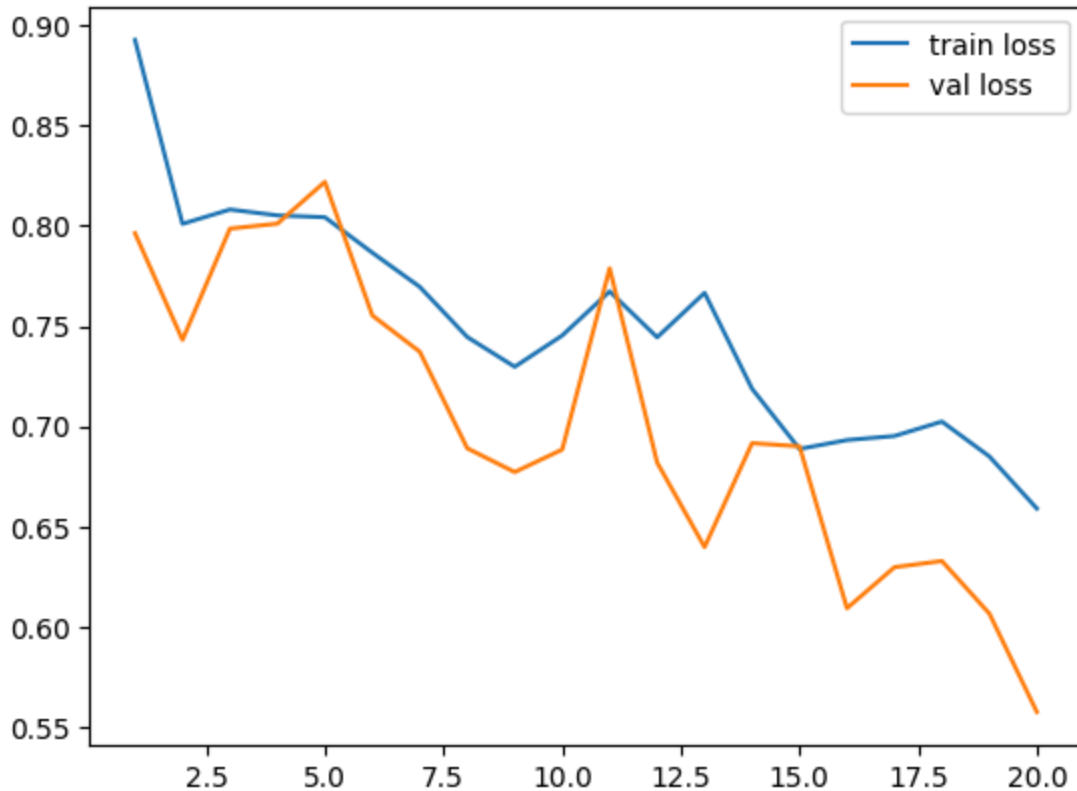


Clipping input data to the valid range for imshow with RGB data ([0..1] for floats or [0..255] for integers). Got range [0.0..1.0996022].



1.7 Plot loss history

```
In [16]: plt.plot(np.arange(1, len(train_losses)+1), train_losses, label='train loss')
plt.plot(np.arange(1, len(val_losses)+1), val_losses, label='val loss')
plt.legend()
plt.show()
```



1.8 (TODO) Plot t-SNE

After the training process is finished, we evaluate whether the learned representation is informative. In this task, embedding visualization is often performed to verify that the feature learned by the network is behaving as intended, i.e., feature vectors of the same identity should be close to each other and far away from other identities. Since the feature vector dimension is too high for a human to interpret, therefore, in this assignment, we use the t-SNE dimensionality reduction technique to compress the feature into a 2D space.

Instructions

TODO 10: Extract the feature vectors of the test set and store them as

`embeddings` : torch.FloatTensor = feature vectors of all images in the test set

`identities` : list or torch.Tensor or np.array = identities of all images in the test set

Hint

Use `FaceDataset` that is imported at `Common Dataset` section

WARNING!! Don't forget load its best weights and change to eval mode first

```
In [17]: test_batch_size = 32

# TODO 10: Extract the feature vectors of the test set and store them as
# `embeddings`: torch.FloatTensor = feature vectors of all images in the test set
```

```

# `identities`: List or torch.Tensor or np.array = identities of all images in the
# Hint      => Use `FaceDataset` that is imported at `Common Dataset` section
# WARNING!! => Don't forget load its best weights and change to eval mode first

face_dataset = FaceDataset(root_dir='large_prepared_data/test', transform=val_trans

face_loader = DataLoader(face_dataset, batch_size=test_batch_size, shuffle=False)

siamese_model.load_state_dict(torch.load(best_weights_path))
siamese_model.to(device)
siamese_model.eval()

embeddings = []
identities = []

with torch.no_grad():
    for img, identity, img_numpy in tqdm(face_loader):
        img = img.to(device)

        output = siamese_model.extract_feature(img)

        embeddings.append(output.cpu())
        identities.extend(identity) # extend because identity return tuples with #b

embeddings = torch.cat(embeddings)

print(f"Embeddings Shape: {embeddings.shape}")

0%|          | 0/10 [00:00<?, ?it/s]
Embeddings Shape: torch.Size([301, 128])

```

```

In [58]: import time
        from sklearn.manifold import TSNE

        time_start = time.time()
        tsne = TSNE(n_components=2, verbose=1, perplexity=30, max_iter=3000)
        tsne_result = tsne.fit_transform(embeddings, identities)
        print('t-SNE done! Time elapsed: {} seconds'.format(time.time()-time_start))

```

```

[t-SNE] Computing 91 nearest neighbors...
[t-SNE] Indexed 301 samples in 0.001s...
[t-SNE] Computed neighbors for 301 samples in 0.013s...
[t-SNE] Computed conditional probabilities for sample 301 / 301
[t-SNE] Mean sigma: 1.148626
[t-SNE] KL divergence after 250 iterations with early exaggeration: 51.958199
[t-SNE] KL divergence after 1250 iterations: 0.504960
t-SNE done! Time elapsed: 0.5066311359405518 seconds

```

```

In [59]: import plotly.express as px

        # relabel to be easier to see in t-SNE visualization
        label = []
        idx = 0
        id2label = dict()
        for identity in identities:
            identity = int(identity)
            if identity not in id2label:

```

```

id2label[identity] = idx
idx += 1
label.append(id2label[identity])

df_subset = pd.DataFrame({'label': label})

df_subset['tsne-2d-one'] = tsne_result[:,0]
df_subset['tsne-2d-two'] = tsne_result[:,1]

fig = px.scatter(df_subset, x="tsne-2d-one", y="tsne-2d-two", color="label", height=
fig.show()

```

1.9 (TODO) Analyzing feature vector visualization result

TODO 11: What could you say about the displayed visualization? Is the model working as expected?

Answer here: **Not as expected. It's kinda random in the visualization.**

Part 2: Face verification with triplet loss + center loss

In contrast to contrastive loss which learns to distinguish whether the two images have the same identity, triplet loss is proposed as an alternative by introducing an anchor image as a third input. Triplet loss receives three inputs: anchor, positive pair, and negative pair. The positive pair is an image having the same identity as the anchor while the negative pair is the one with a different identity. The loss learns to minimize the distance between the anchor and positive pair, and maximize the distance between the anchor and the negative pair. Compared to contrastive loss, triplet loss offers more training stability and better model performance.

Nevertheless, both contrastive and triplet loss also has some shortcomings as these losses only optimize on a pairwise level. This might result in feature vectors of the same identity taking the form of multiple small clusters scattering across the feature space since there is no explicit loss to bind them into a single group. Therefore, a center loss is proposed to mitigate this problem by encouraging the intra-class (same identity) feature vectors to come closer to their intra-class centroids.

In this part, you are going to implement a face verification network by jointly training **three losses**: triplet, center, and cross-entropy loss.

2.1 Dataset and DataLoader

In this section, you are going to implement a dataloader for the combined loss. The dataloader should return the tuple `(anchor_img, pos_img, neg_img, anchor_label, pos_label, neg_label)`.

- The positive image must have the same identity as the anchor image.
- The negative image must have a different identity from the anchor image.

```
In [20]: class TripletDataset(Dataset):
    def __init__(self, root_dir, transform=None):
        np.random.seed(123)
        random.seed(123)
        self.root_dir = root_dir
        self.transform = transform

        label_df = pd.read_csv(f'{root_dir}/label_df.csv')
        label_df = label_df.groupby('identity')['filename'].apply(list).reset_index()
        self.images = []
        self.labels = []
        self.label2indices = dict()
        self.load_images_to_memory_(label_df)

    def load_images_to_memory_(self, label_df):
        # Load images and labels into memory
        # We have to relabel from identities to 0,1,2,...,num_classes-1
        # Relevant variables
        # 1. `self.images` = PIL images (Also convert it to RGB)
        # 2. `self.label2indices` = the dictionary storing a label as a key and the i
        # 3. `self.labels` = labels of images (relabeling)
        # Note: the index of images is from iterating over label_df
        self.images = []
        self.label2indices = defaultdict(list)
        self.labels = []
        img_idx = 0
        label_idx = 0
        identity2label = dict()
        for idx in tqdm(range(len(label_df))):
            row = label_df.iloc[idx]
            identity, filenames = row['identity'], row['filenames']

            if identity not in identity2label:
                identity2label[identity] = label_idx
                label_idx += 1

            label = identity2label[identity]

            for filename in filenames:
                self.images.append(Image.open(f'{self.root_dir}/{filename}').convert('RGB'))
                self.labels.append(label)
                self.label2indices[label].append(img_idx)
                img_idx += 1

    def __len__(self):
        return len(self.images)
```

```

def __getitem__(self, idx):
    # generate an anchor image, a positive image and a negative image together wi
    # anchor image is the image according to idx
    # positive image is the image that has the same identity with the anchor imag
    # negative image is the image that has the different identity with the anchor
    anchor_img = self.images[idx]
    anchor_label = self.labels[idx]

    pos_idx = random.choice(self.label2indices[anchor_label])
    while pos_idx == idx:
        pos_idx = random.choice(self.label2indices[anchor_label])
    pos_img = self.images[pos_idx]
    pos_label = anchor_label
    neg_label = random.choice(list(set(self.labels) - {anchor_label}))
    neg_img = self.images[random.choice(self.label2indices[neg_label])]

    # utilize `self.transform` to convert images to tensors
    if self.transform is not None:
        anchor_img = self.transform(anchor_img)
        pos_img = self.transform(pos_img)
        neg_img = self.transform(neg_img)

    return anchor_img, pos_img, neg_img, anchor_label, pos_label, neg_label

```

```

In [21]: image_size = 224
train_transform = transforms.Compose([
    transforms.Resize((256, 256)),
    transforms.CenterCrop(image_size),
    transforms.RandomHorizontalFlip(p=0.5),
    transforms.ToTensor(),
    transforms.Normalize(mean=[0.5319, 0.4399, 0.3929],
                        std=[0.3076, 0.2898, 0.2907])
])

val_transform = transforms.Compose([
    transforms.Resize((256, 256)),
    transforms.CenterCrop(image_size),
    transforms.ToTensor(),
    transforms.Normalize(mean=[0.5319, 0.4399, 0.3929],
                        std=[0.3076, 0.2898, 0.2907])
])

train_batch_size = 16
val_batch_size = 16
test_batch_size = 16

# declare the datasets and the dataloaders

train_triplet_dataset = TripletDataset(root_dir='large_prepared_data/train', transf
train_triplet_dataloader = DataLoader(train_triplet_dataset, batch_size=train_batch

val_triplet_dataset = TripletDataset(root_dir='large_prepared_data/val', transform=
val_triplet_dataloader = DataLoader(val_triplet_dataset, batch_size=val_batch_size,

```



```
test_triplet_dataset = TripletDataset(root_dir='large_prepared_data/test', transform=
test_triplet_dataloader = DataLoader(test_triplet_dataset, batch_size=test_batch_si
```

```
0%|          | 0/80 [00:00<?, ?it/s]
0%|          | 0/10 [00:00<?, ?it/s]
0%|          | 0/10 [00:00<?, ?it/s]
```

2.2 Triplet Network

This network is a simplified version of strong baseline person re-identification task (ref: <https://arxiv.org/abs/1903.07071>)

Triplet network consists of 3 modules

1. Feature extractor (ResNet18) is for mapping an image to a feature map
2. Global pooling (Global average pooling or use flatten instead) is for converting a feature map to a feature vector
3. Bottleneck (Batch normalization only scale not shift) is for the consistency of training between triplet loss and crossentropy because it is difficult to optimize those losses in embedding space at the same time
4. Linear classifier is to classify who this face is

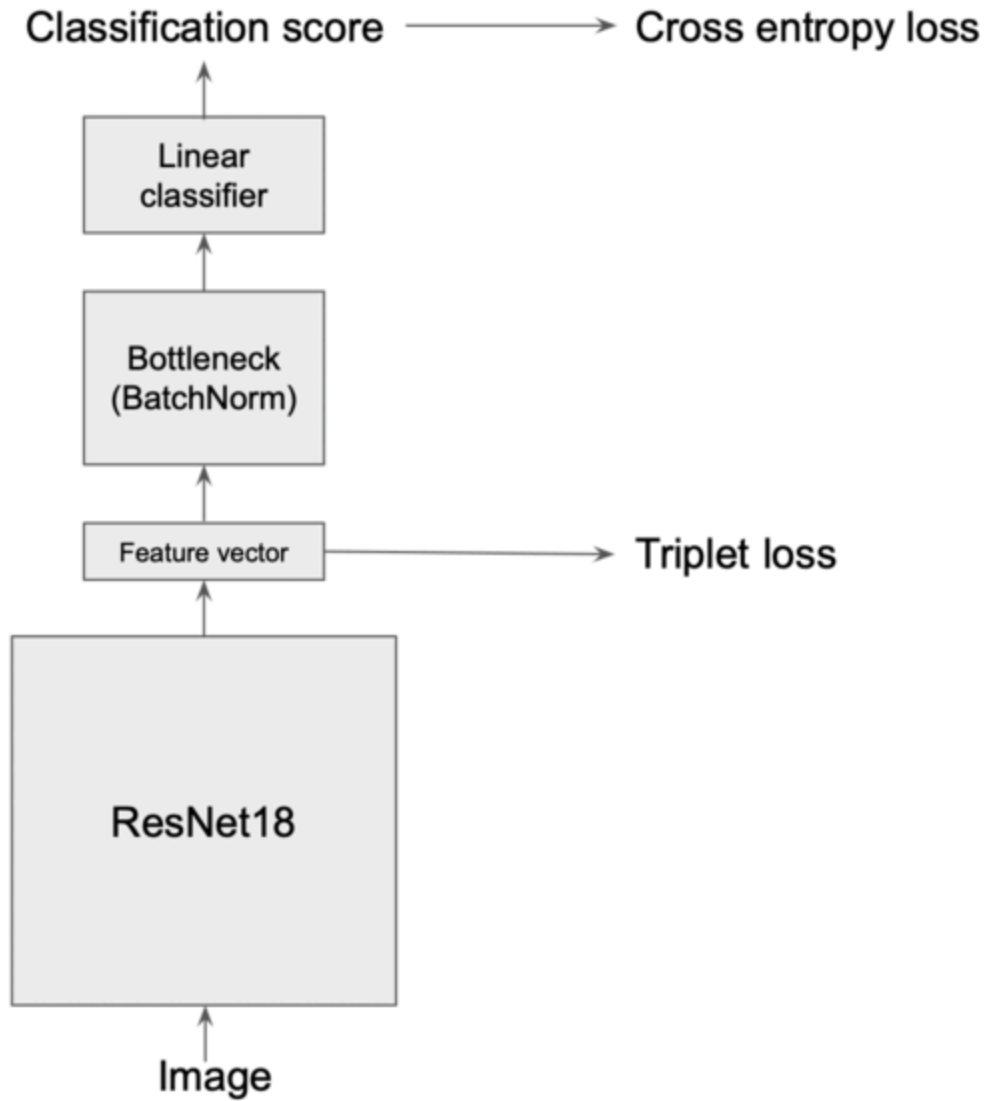
Note

1. ResNet18 can call via `torchvision` library (How to use torchvision: <https://pytorch.org/vision/stable/models.html>)
2. ResNet18 we use will not load pretrained weights
3. We will use ResNet18 only extracting feature maps.

Hint!

1. For global average pooling, read the documentation of `nn.AdaptiveAvgPool1d`
2. For bottleneck, try `requires_grad_` method to stop calculating gradient at batch normalization

Figure 3 Triplet network



```

In [22]: def weights_init_kaiming(m):
    classname = m.__class__.__name__
    if m.affine:
        nn.init.constant_(m.weight, 1.0)
        nn.init.constant_(m.bias, 0.0)

    def weights_init_classifier(m):
        classname = m.__class__.__name__
        if classname.find('Linear') != -1:
            nn.init.normal_(m.weight, std=0.001)
            if m.bias:
                nn.init.constant_(m.bias, 0.0)

    class TripletNetwork(nn.Module):
        # implement the triplet network
        def __init__(self, num_classes):
            super(TripletNetwork, self).__init__()
            resnet = torchvision.models.resnet18(pretrained=False)
            self.conv = torch.nn.Sequential(*list(resnet.children())[:-1])
            self.global_pool = nn.AdaptiveAvgPool2d(1)
  
```

```

self.bottleneck = nn.BatchNorm1d(512)
# no shift
self.bottleneck.bias.requires_grad_(False)

self.classifier = nn.Linear(512, num_classes, bias=False)

self.bottleneck.apply(weights_init_kaiming)
self.classifier.apply(weights_init_classifier)

def extract_feature(self, x):
    x = self.conv(x)

    global_feat = self.global_pool(x)
    global_feat = global_feat.view(global_feat.size(0), -1)

    feat = self.bottleneck(global_feat)

    if not self.training:
        return feat

    cls_score = self.classifier(feat)

    return global_feat, cls_score

def forward(self, anchor_img, pos_img, neg_img):
    anchor_feat, anchor_score = self.extract_feature(anchor_img)
    pos_feat, pos_score = self.extract_feature(pos_img)
    neg_feat, neg_score = self.extract_feature(neg_img)

    feats = torch.cat((anchor_feat, pos_feat, neg_feat))
    scores = torch.cat((anchor_score, pos_score, neg_score))
    return anchor_feat, pos_feat, neg_feat, feats, scores

```

2.3 Triplet loss

Triplet loss uses the same concept as contrastive loss that is the anchor image will pull the positive one closer and push the negative one further than the constant margin away. Hence, we have sampled three images to compute it. The three images consist of:

1. an anchor image = an initial image
2. a positive image = the image having the same identity as the anchor image
3. a negative image = the image having a different identity from the anchor image

The triplet loss is mathematically defined as:

$$L_{Triplet} = \max(0, m + d(r_a, r_p) - d(r_a, r_n))$$

where

- $d(r_1, r_2)$ = euclidean distance between r_1 and r_2
- m = margin
- r_a = the feature vector of the anchor image

- r_p = the feature vector of the positive image
- r_n = the feature vector of the negative image

The loss aims to minimize $d(r_a, r_p)$ while maximizing $d(r_a, r_n)$ at the same time.

The margin m is used to prevent the loss from collapsing to a trivial solution. For instance, when $m = 0$, the model could achieve $L = 0$ just by exploiting the objective by setting $r_a = r_p = r_n$, and the model, as a result, would learn nothing useful.

```
In [23]: class TripletLoss(nn.Module):
# implement the triplet loss
def __init__(self, margin=2.0):
    super(TripletLoss, self).__init__()
    self.margin = margin
    self.euclidean_dist_fn = nn.PairwiseDistance(p=2)

def forward(self, anchor_img, pos_img, neg_img):
    pos_dist = self.euclidean_dist_fn(anchor_img, pos_img)
    neg_dist = self.euclidean_dist_fn(anchor_img, neg_img)
    return torch.mean(torch.relu(pos_dist - neg_dist + self.margin))
```

2.4 Center loss

Reference: <https://ydwen.github.io/papers/WenECCV16.pdf>

Definition

Centroid: a representative of each class. There are several ways to select a representative, one way to do this is to take an average on those embeddings.

Concept

Center loss enforces the intra-class feature vectors to come closer to the centroid of their class so that all feature vectors of the same identity are clustered around a single centroid. Since the model is updated after each iteration, the centroids should also be updated accordingly. However, re-calculating the centroids by averaging the feature vectors for each class every iteration is computational-extensive on a large scale. Therefore, the centroid is instead learned from the representative of each class from the sampled data.

Implementation detail

In `__init__` method

1. For simplicity, we will store the centroids in the class `CenterLoss`. Therefore, you have to initialize the centroids as a random tensor with the size of `(num_classes, feature_dimension)`. The tensor has to be set with `nn.Parameter` so that the gradient could be calculated (ref: <https://pytorch.org/docs/stable/generated/torch.nn.parameter.Parameter.html>).

In the `forward` method

1. Calculate the distance between the feature vector and its center with a squared Euclidean distance
2. Clip the value in each element to be not greater than $1e+12$, not lower than $1e-12$, and sum them
3. Normalize the loss with its batch size

Center loss equation

$$L_{Center} = \frac{1}{B} \sum_{j=1}^B \|f_j - c_{y_j}\|_2^2$$

where

- f_j = a feature vector before fed into the bottleneck at index j
- c_{y_j} = a center of the class corresponded to the index j
- B = batch size

The center loss minimized the distance between the feature vector and the centroid of the corresponded identity.

Note

1. At `__init__` method in step 1. Do not forget to transfer the parameters to GPU with `.cuda()` otherwise, it will not utilize GPU on this part.
2. Clip the magnitude of the loss at the final stage before updating both upper bound and lower bound to avoid vanishing / exploding gradients.

Hint

1. In step 2 of the `forward` method, you should use `torch.clamp` (ref: <https://pytorch.org/docs/stable/generated/torch.clamp.html>) to clip the lower bound and the upper bound as you want.

```
In [24]: class CenterLoss(nn.Module):
def __init__(self, num_classes, feat_dim):
    super(CenterLoss, self).__init__()
    self.device = 'cuda' if torch.cuda.is_available() else 'cpu'
    self.num_classes = num_classes
    self.feat_dim = feat_dim
    self.centers = nn.Parameter(torch.randn(self.num_classes, self.feat_dim).to(self.device))

def forward(self, x, labels):
    batch_size = x.size(0)
    # compute squared euclidean distance
    distmat = torch.pow(x, 2).sum(dim=1, keepdim=True).expand(batch_size, self.num_classes) + \
        torch.pow(self.centers, 2).sum(dim=1, keepdim=True).expand(self.num_classes, 1)
    distmat.addmm_(1, -2, x, self.centers.t())

    classes = torch.arange(self.num_classes).long().to(self.device)
```

```

labels = labels.unsqueeze(1).expand(batch_size, self.num_classes)
mask = labels.eq(classes.expand(batch_size, self.num_classes))

dist = distmat * mask.float()
loss = dist.clamp(min=1e-12, max=1e+12).sum() / batch_size
return loss

```

```

In [25]: center_loss_fn = CenterLoss(80, 512)
feats = torch.randn(32, 512).cuda()
labels = torch.randn(32).cuda()
center_loss_fn(feats, labels)

```

C:\Users\Faith\AppData\Local\Temp\ipykernel_13952\782439988.py:14: UserWarning:

This overload of addmm_ is deprecated:

```
addmm_(Number beta, Number alpha, Tensor mat1, Tensor mat2)
```

Consider using one of the following signatures instead:

```
addmm_(Tensor mat1, Tensor mat2, *, Number beta = 1, Number alpha = 1) (Triggered internally at C:\actions-runner\work\pytorch\pytorch\pytorch\torch\src\utils\python_arg_parser.cpp:1805.)
```

```
Out[25]: tensor(8.0000e-11, device='cuda:0', grad_fn=<DivBackward0>)
```

2.5 Declare model, criterions, optimizers, hyparameters and scheduler

```

In [26]: # for triplet loss
triplet_margin = 3.0

# get the number of classes to construct a linear classifier and the parameters in
num_classes = len(set(train_triplet_dataset.labels))
print(f'num_classes = {num_classes}')

# declare the triplet model and the triplet criterion
triplet_model = TripletNetwork(num_classes)
triplet_criterion = TripletLoss(margin=triplet_margin)

triplet_optimizer = optim.Adam(triplet_model.parameters(), lr=5e-4)
triplet_scheduler = optim.lr_scheduler.ReduceLROnPlateau(triplet_optimizer, 'min',

# declare cross entropy loss
crossentropy_criterion = nn.CrossEntropyLoss()

# For center loss
# declare the center criterion
triplet_center_criterion = CenterLoss(num_classes, 512)

triplet_center_optimizer = optim.Adam(triplet_center_criterion.parameters(), lr=0.5)
triplet_center_loss_weight = 5e-2

```



```
c:\Users\Faith\miniconda3\envs\pattern\lib\site-packages\torchvision\models\_utils.p
y:208: UserWarning:
```

The parameter 'pretrained' is deprecated since 0.13 and may be removed in the future, please use 'weights' instead.

```
c:\Users\Faith\miniconda3\envs\pattern\lib\site-packages\torchvision\models\_utils.p
y:223: UserWarning:
```

Arguments other than a weight enum or `None` for 'weights' are deprecated since 0.13 and may be removed in the future. The current behavior is equivalent to passing `weights=None`.

```
num_classes = 80
```

2.6 Training loop

The figure below shows how the center loss is updated in the original paper. L_s , L_c , W , θ_c and c_j stand for classification loss, center loss, classification head's weight, CNN weight, and centroid of the class j , respectively. Line 6 shows how centroids are updated.

Algorithm 1. The discriminative feature learning algorithm

Input: Training data $\{x_i\}$. Initialized parameters θ_C in convolution layers. Parameters W and $\{c_j | j = 1, 2, \dots, n\}$ in loss layers, respectively. Hyperparameter λ , α and learning rate μ^t . The number of iteration $t \leftarrow 0$.

Output: The parameters θ_C .

1: **while** not converge **do**

2: $t \leftarrow t + 1$.

3: Compute the joint loss by $\mathcal{L}^t = \mathcal{L}_s^t + \mathcal{L}_c^t$.

4: Compute the backpropagation error $\frac{\partial \mathcal{L}^t}{\partial x_i^t}$ for each i by $\frac{\partial \mathcal{L}^t}{\partial x_i^t} = \frac{\partial \mathcal{L}_s^t}{\partial x_i^t} + \lambda \cdot \frac{\partial \mathcal{L}_c^t}{\partial x_i^t}$.

5: Update the parameters W by $W^{t+1} = W^t - \mu^t \cdot \frac{\partial \mathcal{L}^t}{\partial W^t} = W^t - \mu^t \cdot \frac{\partial \mathcal{L}_s^t}{\partial W^t}$.

6: Update the parameters c_j for each j by $c_j^{t+1} = c_j^t - \alpha \cdot \Delta c_j^t$.

7: Update the parameters θ_C by $\theta_C^{t+1} = \theta_C^t - \mu^t \sum_i^m \frac{\partial \mathcal{L}^t}{\partial x_i^t} \cdot \frac{\partial x_i^t}{\partial \theta_C^t}$.

8: **end while**

Joint loss equation

$$L_{Joint} = L_{ID} + L_{Triplet} + \beta L_{Center}$$

where

- L_{Joint} = joint loss
- L_{ID} = identity loss (cross entropy loss in this implementation)
- $L_{Triplet}$ = triplet loss
- L_{Center} = center loss
- β = center loss weights (affects only the triplet network, not center update)

The training procedure of the triplet network consists of the following steps:

1. Forward pass the `anchor_img`
2. Forward pass the `positive_img`
3. Forward pass the `negative_img`
4. Calculate the distance between the feature vectors of `anchor_img`, `positive_img` and `negative_img` and calculate a triplet loss ($L_{Triplet}$).
5. Calculate the classification loss (L_{ID}).
6. Calculate the center loss (L_{Center}).
7. Scale the center loss with the predetermined weight (β) (only used to update the triplet network not to update the centroids).
8. Sum the losses from steps 4-7 and update the triplet network. (Line 3-5)
9. Rescale the gradients with $\frac{1}{\beta}$ at the loss in step 7 and update the centroids. (Line 6-7)
10. Repeat steps 1-9 till converge

As you would notice, steps 1- 3 share the same model weight for feature extraction. Thus, the word "triplet" in triplet loss originates in a similar fashion as the "Siamese twins" but this loss function utilizes three feature vectors simultaneously instead of two.

Hint

1. At the gradient rescaling for the centroids, we should iterate parameters in center loss first with `.parameters()` and adjust the property of each parameter with `.grad.data`

```
In [27]: num_epochs = 20
device = "cuda" if torch.cuda.is_available() else "cpu"

os.makedirs('weights', exist_ok=True)
best_weights_path = 'weights/best_triplet_weights.pth'
triplet_model.to(device)
min_val_loss = float('inf')

train_losses = []
train_triplet_losses = []
train_crossentropy_losses = []
train_center_losses = []

val_losses = []
val_triplet_losses = []

for epoch in tqdm(range(num_epochs)):
    triplet_model.train()
    total_train_loss = 0
    total_train_triplet_loss = 0
    total_train_crossentropy_loss = 0
    total_train_center_loss = 0
    for anchor_img, pos_img, neg_img, anchor_label, pos_label, neg_label in tqdm(tr

        # feed data to the triplet model and compute triplet loss
        anchor_img, pos_img, neg_img = anchor_img.to(device), pos_img.to(device), n
        anchor_label, pos_label, neg_label = anchor_label.to(device), pos_label.to(
```

```

labels = torch.cat((anchor_label, pos_label, neg_label))

anchor_feat, pos_feat, neg_feat, feats, scores = triplet_model(anchor_img,
train_triplet_loss = triplet_criterion(anchor_feat, pos_feat, neg_feat)

# compute cross entropy loss
train_crossentropy_loss = crossentropy_criterion(scores, labels)

# compute center loss
train_center_loss = triplet_center_loss_weight * triplet_center_criterion(f

train_loss = train_triplet_loss + train_crossentropy_loss + train_center_lo
total_train_loss += train_loss.item()
total_train_triplet_loss += train_triplet_loss.item()
total_train_crossentropy_loss += train_crossentropy_loss.item()
total_train_center_loss += train_center_loss.item()

# set zero gradients at two optimizers
triplet_optimizer.zero_grad()
triplet_center_optimizer.zero_grad()

# back propagate at triplet network and step the main optimizer
train_loss.backward()
triplet_optimizer.step()

# rescale gradients of centers because `triplet_center_loss_weight` should
# and step the center optimizer
for param in triplet_center_criterion.parameters():
    param.grad.data *= (1. / triplet_center_loss_weight)
triplet_center_optimizer.step()

current_train_loss = total_train_loss / len(train_triplet_dataloader)
current_train_triplet_loss = total_train_triplet_loss / len(train_triplet_datal
current_train_crossentropy_loss = total_train_crossentropy_loss / len(train_tri
current_train_center_loss = total_train_center_loss / len(train_triplet_datalea
train_losses.append(current_train_loss)
train_triplet_losses.append(current_train_triplet_loss)
train_crossentropy_losses.append(current_train_crossentropy_loss)
train_center_losses.append(current_train_center_loss)

total_val_loss = 0
total_val_triplet_loss = 0
triplet_model.eval()
for val_anchor_img, val_pos_img, val_neg_img, _, _, _ in val_triplet_dataloader

# feed data to the triplet model and compute triplet loss
val_anchor_img, val_pos_img, val_neg_img = val_anchor_img.to(device), val_pos
with torch.no_grad():
    val_anchor_feat = triplet_model.extract_feature(val_anchor_img)
    val_pos_feat = triplet_model.extract_feature(val_pos_img)
    val_neg_feat = triplet_model.extract_feature(val_neg_img)
    val_triplet_loss = triplet_criterion(val_anchor_feat, val_pos_feat, val_n

val_loss = val_triplet_loss

```

```

    total_val_loss += val_loss.item()
    total_val_triplet_loss += val_triplet_loss.item()
    current_val_loss = total_val_loss / len(val_triplet_dataloader)
    current_val_triplet_loss = total_val_triplet_loss / len(val_triplet_dataloader)
    val_losses.append(current_val_loss)
    val_triplet_losses.append(current_val_triplet_loss)

    if current_val_loss < min_val_loss:
        min_val_loss = current_val_loss
        torch.save(triplet_model.state_dict(), best_weights_path)
    print(f'Epoch {epoch+1} - Train loss = {current_train_loss:.4f} - Train triplet
triplet_scheduler.step(current_val_loss)

```

```

0%|          | 0/20 [00:00<?, ?it/s]
0%|          | 0/153 [00:00<?, ?it/s]
Epoch 1 - Train loss = 10.9075 - Train triplet loss = 2.1489 - Train crossentropy loss = 3.8459 - Train center loss = 4.9126 - Val loss = 1.8157 - Val triplet loss = 1.8157 - best min_val_loss = 1.8157 - lr = 0.00050000
0%|          | 0/153 [00:00<?, ?it/s]
Epoch 2 - Train loss = 6.9753 - Train triplet loss = 1.5537 - Train crossentropy loss = 2.7780 - Train center loss = 2.6436 - Val loss = 1.3660 - Val triplet loss = 1.3660 - best min_val_loss = 1.3660 - lr = 0.00050000
0%|          | 0/153 [00:00<?, ?it/s]
Epoch 3 - Train loss = 6.1725 - Train triplet loss = 1.2433 - Train crossentropy loss = 1.9275 - Train center loss = 3.0017 - Val loss = 1.0468 - Val triplet loss = 1.0468 - best min_val_loss = 1.0468 - lr = 0.00050000
0%|          | 0/153 [00:00<?, ?it/s]
Epoch 4 - Train loss = 5.8161 - Train triplet loss = 1.1433 - Train crossentropy loss = 1.2139 - Train center loss = 3.4590 - Val loss = 1.2399 - Val triplet loss = 1.2399 - best min_val_loss = 1.0468 - lr = 0.00050000
0%|          | 0/153 [00:00<?, ?it/s]
Epoch 5 - Train loss = 5.0842 - Train triplet loss = 1.0076 - Train crossentropy loss = 0.8118 - Train center loss = 3.2647 - Val loss = 0.9530 - Val triplet loss = 0.9530 - best min_val_loss = 0.9530 - lr = 0.00050000
0%|          | 0/153 [00:00<?, ?it/s]
Epoch 6 - Train loss = 4.8446 - Train triplet loss = 0.7910 - Train crossentropy loss = 0.4764 - Train center loss = 3.5772 - Val loss = 0.9451 - Val triplet loss = 0.9451 - best min_val_loss = 0.9451 - lr = 0.00050000
0%|          | 0/153 [00:00<?, ?it/s]
Epoch 7 - Train loss = 3.9858 - Train triplet loss = 0.7187 - Train crossentropy loss = 0.2715 - Train center loss = 2.9956 - Val loss = 1.0832 - Val triplet loss = 1.0832 - best min_val_loss = 0.9451 - lr = 0.00050000
0%|          | 0/153 [00:00<?, ?it/s]
Epoch 8 - Train loss = 4.0916 - Train triplet loss = 0.6948 - Train crossentropy loss = 0.2126 - Train center loss = 3.1842 - Val loss = 0.8105 - Val triplet loss = 0.8105 - best min_val_loss = 0.8105 - lr = 0.00050000
0%|          | 0/153 [00:00<?, ?it/s]
Epoch 9 - Train loss = 3.6798 - Train triplet loss = 0.6291 - Train crossentropy loss = 0.1343 - Train center loss = 2.9165 - Val loss = 0.7560 - Val triplet loss = 0.7560 - best min_val_loss = 0.7560 - lr = 0.00050000
0%|          | 0/153 [00:00<?, ?it/s]
Epoch 10 - Train loss = 3.6155 - Train triplet loss = 0.5528 - Train crossentropy loss = 0.1187 - Train center loss = 2.9440 - Val loss = 0.7834 - Val triplet loss = 0.7834 - best min_val_loss = 0.7560 - lr = 0.00050000
0%|          | 0/153 [00:00<?, ?it/s]

```

```

Epoch 11 - Train loss = 4.0656 - Train triplet loss = 0.5319 - Train crossentropy loss = 0.0746 - Train center loss = 3.4591 - Val loss = 0.8310 - Val triplet loss = 0.8310 - best min_val_loss = 0.7560 - lr = 0.00050000
0%|          | 0/153 [00:00<?, ?it/s]
Epoch 12 - Train loss = 3.4307 - Train triplet loss = 0.4709 - Train crossentropy loss = 0.0691 - Train center loss = 2.8907 - Val loss = 1.0499 - Val triplet loss = 1.0499 - best min_val_loss = 0.7560 - lr = 0.00050000
0%|          | 0/153 [00:00<?, ?it/s]
Epoch 13 - Train loss = 3.4401 - Train triplet loss = 0.4559 - Train crossentropy loss = 0.0519 - Train center loss = 2.9323 - Val loss = 0.9076 - Val triplet loss = 0.9076 - best min_val_loss = 0.7560 - lr = 0.00050000
0%|          | 0/153 [00:00<?, ?it/s]
Epoch 14 - Train loss = 4.5415 - Train triplet loss = 0.3970 - Train crossentropy loss = 0.0536 - Train center loss = 4.0909 - Val loss = 0.7449 - Val triplet loss = 0.7449 - best min_val_loss = 0.7449 - lr = 0.00050000
0%|          | 0/153 [00:00<?, ?it/s]
Epoch 15 - Train loss = 4.9330 - Train triplet loss = 0.3346 - Train crossentropy loss = 0.0517 - Train center loss = 4.5468 - Val loss = 1.0060 - Val triplet loss = 1.0060 - best min_val_loss = 0.7449 - lr = 0.00050000
0%|          | 0/153 [00:00<?, ?it/s]
Epoch 16 - Train loss = 4.2588 - Train triplet loss = 0.3189 - Train crossentropy loss = 0.0440 - Train center loss = 3.8958 - Val loss = 0.8254 - Val triplet loss = 0.8254 - best min_val_loss = 0.7449 - lr = 0.00050000
0%|          | 0/153 [00:00<?, ?it/s]
Epoch 17 - Train loss = 2.9301 - Train triplet loss = 0.2925 - Train crossentropy loss = 0.0280 - Train center loss = 2.6096 - Val loss = 0.7008 - Val triplet loss = 0.7008 - best min_val_loss = 0.7008 - lr = 0.00050000
0%|          | 0/153 [00:00<?, ?it/s]
Epoch 18 - Train loss = 3.8484 - Train triplet loss = 0.3103 - Train crossentropy loss = 0.0233 - Train center loss = 3.5148 - Val loss = 0.8103 - Val triplet loss = 0.8103 - best min_val_loss = 0.7008 - lr = 0.00050000
0%|          | 0/153 [00:00<?, ?it/s]
Epoch 19 - Train loss = 3.2196 - Train triplet loss = 0.3101 - Train crossentropy loss = 0.0301 - Train center loss = 2.8794 - Val loss = 0.9854 - Val triplet loss = 0.9854 - best min_val_loss = 0.7008 - lr = 0.00050000
0%|          | 0/153 [00:00<?, ?it/s]
Epoch 20 - Train loss = 3.2337 - Train triplet loss = 0.2978 - Train crossentropy loss = 0.0314 - Train center loss = 2.9045 - Val loss = 0.7909 - Val triplet loss = 0.7909 - best min_val_loss = 0.7008 - lr = 0.00050000

```

2.7 Visualization

The visualization below displays an anchor, positive, and negative image and their respective distance.

```

In [28]: # Showing images
def imshow(img, text=None):
    npimg = img.numpy()
    plt.axis("off")
    if text:
        plt.text(20, 8, text, style='italic', fontweight='bold',
                 bbox={'facecolor': 'white', 'alpha': 0.8, 'pad': 10})

    plt.imshow(np.transpose(npimg, (1, 2, 0)))

```

```

plt.show()

best_weights_path = 'weights/best_triplet_weights.pth'
triplet_model.to(device)
triplet_model.load_state_dict(torch.load(best_weights_path))
triplet_model.eval()

test_anchor_imgs, test_pos_imgs, test_neg_imgs, test_anchor_ids, _, _ = next(iter(t
test_anchor_imgs, test_pos_imgs, test_neg_imgs = test_anchor_imgs.to(device), test_
with torch.no_grad():
    test_anchor_feats = triplet_model.extract_feature(test_anchor_imgs)
    test_pos_feats = triplet_model.extract_feature(test_pos_imgs)
    test_neg_feats = triplet_model.extract_feature(test_neg_imgs)

class UnNormalize(object):
    def __init__(self, mean, std):
        self.mean = mean
        self.std = std

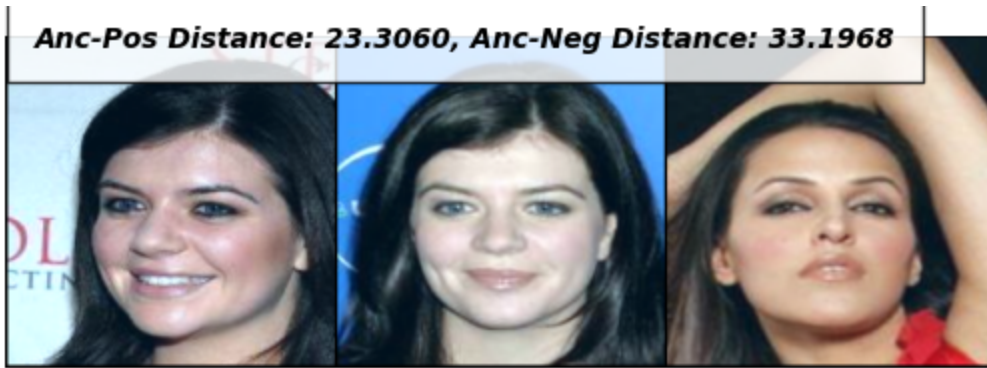
    def __call__(self, tensor):
        """
        Args:
            tensor (Tensor): Tensor image of size (C, H, W) to be normalized.
        Returns:
            Tensor: Normalized image.
        """
        for t, m, s in zip(tensor, self.mean, self.std):
            t.mul_(s).add_(m)
            # The normalize code -> t.sub_(m).div_(s)
        return tensor

unnormalizer = UnNormalize(mean=[0.5319, 0.4399, 0.3929],
                           std=[0.3076, 0.2898, 0.2907])

zip_test_data = zip(test_anchor_feats, test_pos_feats, test_neg_feats, test_anchor_
for test_anchor_feat, test_pos_feat, test_neg_feat, test_anchor_img, test_pos_img,
    test_anchor_img = unnormalizer(test_anchor_img.detach().cpu().unsqueeze(0))
    test_pos_img = unnormalizer(test_pos_img.detach().cpu().unsqueeze(0))
    test_neg_img = unnormalizer(test_neg_img.detach().cpu().unsqueeze(0))
    concatenated = torch.cat((test_anchor_img, test_pos_img, test_neg_img), 0)
    anc_pos_distance = F.pairwise_distance(test_anchor_feat.unsqueeze(0), test_pos_
    anc_neg_distance = F.pairwise_distance(test_anchor_feat.unsqueeze(0), test_neg_
    imshow(torchvision.utils.make_grid(concatenated), f'Anc-Pos Distance: {anc_pos_

```

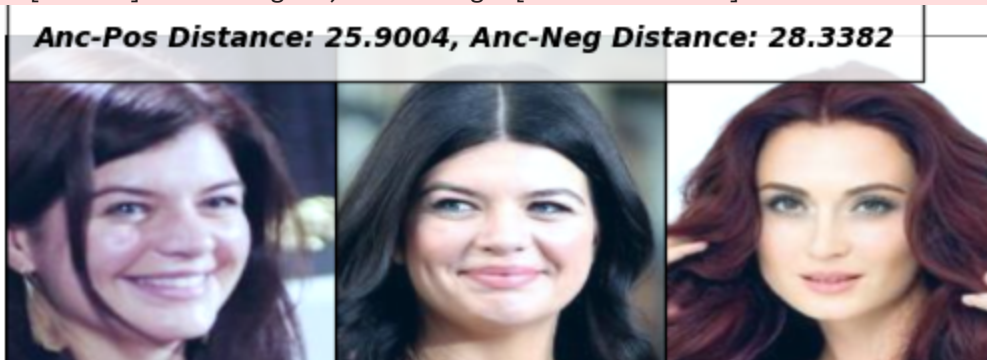
Clipping input data to the valid range for imshow with RGB data ([0..1] for floats or [0..255] for integers). Got range [0.0..1.1742941].



Clipping input data to the valid range for imshow with RGB data ([0..1] for floats or [0..255] for integers). Got range [0.0..1.1742941].



Clipping input data to the valid range for imshow with RGB data ([0..1] for floats or [0..255] for integers). Got range [0.0..1.1742941].



Clipping input data to the valid range for imshow with RGB data ([0..1] for floats or [0..255] for integers). Got range [0.0..1.1659949].



Clipping input data to the valid range for imshow with RGB data ([0..1] for floats or [0..255] for integers). Got range [0.0..1.1327987].



Clipping input data to the valid range for imshow with RGB data ([0..1] for floats or [0..255] for integers). Got range [0.0..1.1742941].



Clipping input data to the valid range for imshow with RGB data ([0..1] for floats or [0..255] for integers). Got range [0.0..1.1742941].



Clipping input data to the valid range for imshow with RGB data ([0..1] for floats or [0..255] for integers). Got range [0.0..1.1742941].



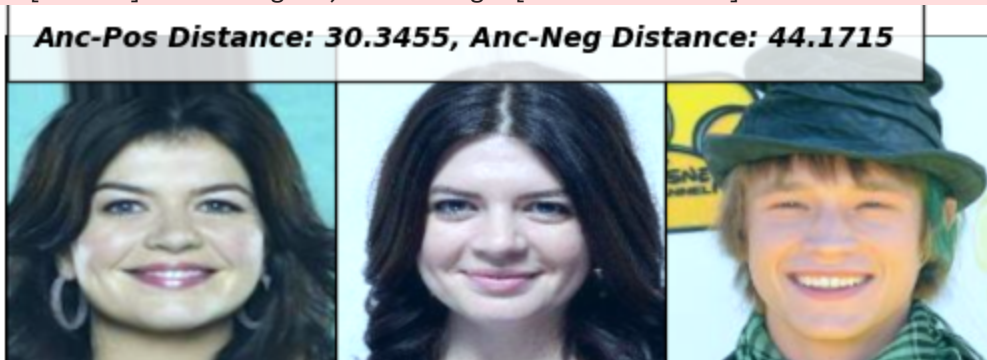
Clipping input data to the valid range for imshow with RGB data ([0..1] for floats or [0..255] for integers). Got range [0.0..1.1701446].



Clipping input data to the valid range for imshow with RGB data ([0..1] for floats or [0..255] for integers). Got range [0.0..1.1742941].



Clipping input data to the valid range for imshow with RGB data ([0..1] for floats or [0..255] for integers). Got range [0.0..1.1742941].



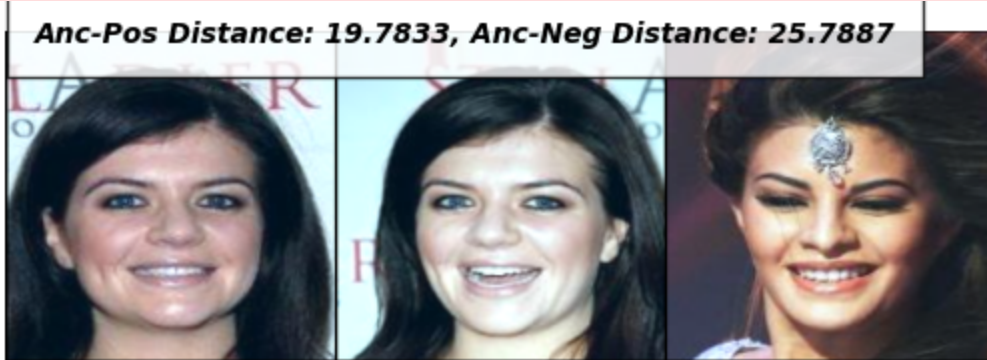
Clipping input data to the valid range for imshow with RGB data ([0..1] for floats or [0..255] for integers). Got range [0.0..1.1162004].



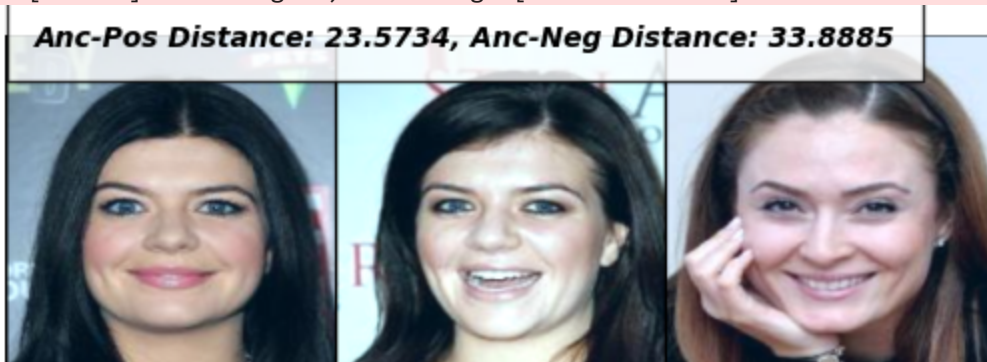
Clipping input data to the valid range for imshow with RGB data ([0..1] for floats or [0..255] for integers). Got range [0.0..1.1742941].



Clipping input data to the valid range for imshow with RGB data ([0..1] for floats or [0..255] for integers). Got range [0.0..1.1742941].



Clipping input data to the valid range for imshow with RGB data ([0..1] for floats or [0..255] for integers). Got range [0.0..1.1742941].

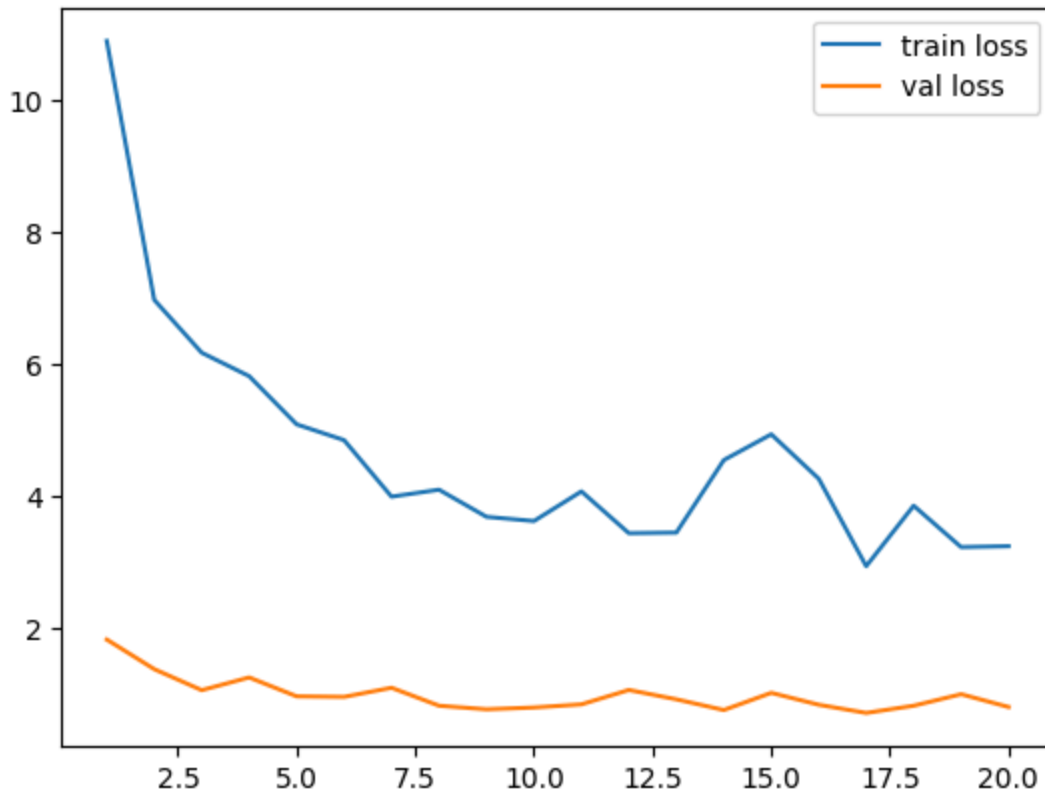


Clipping input data to the valid range for imshow with RGB data ([0..1] for floats or [0..255] for integers). Got range [0.0..1.1742941].

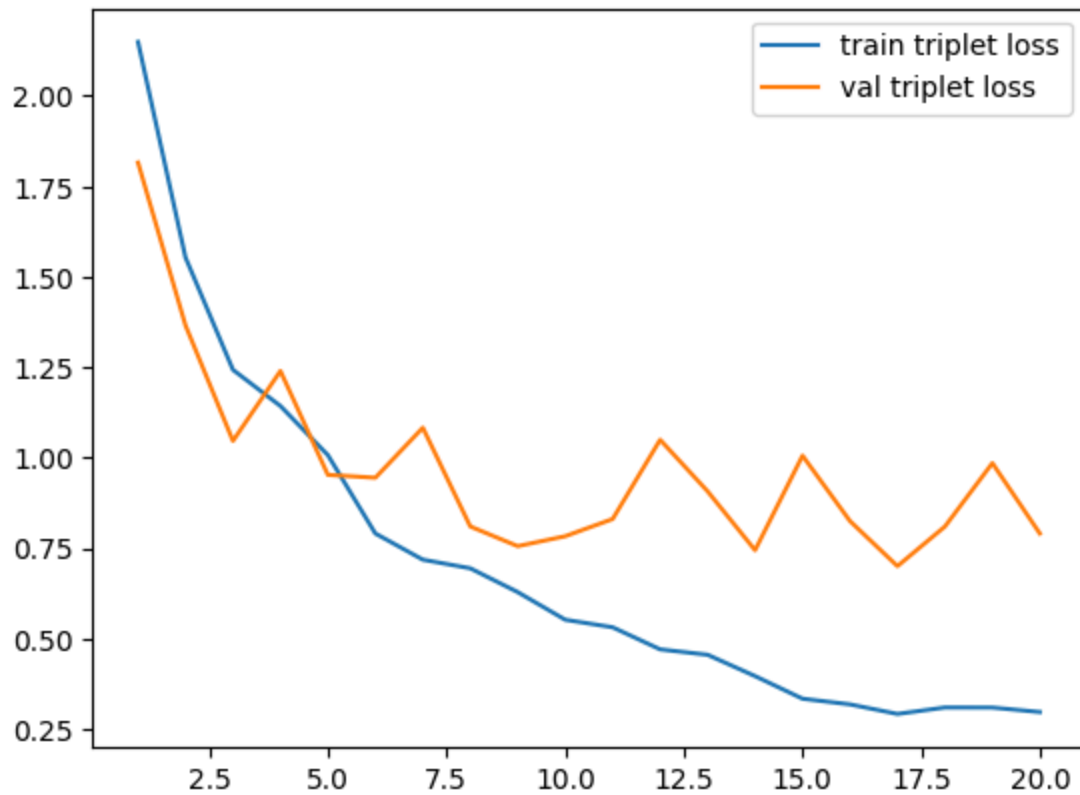


2.8 Plot loss history

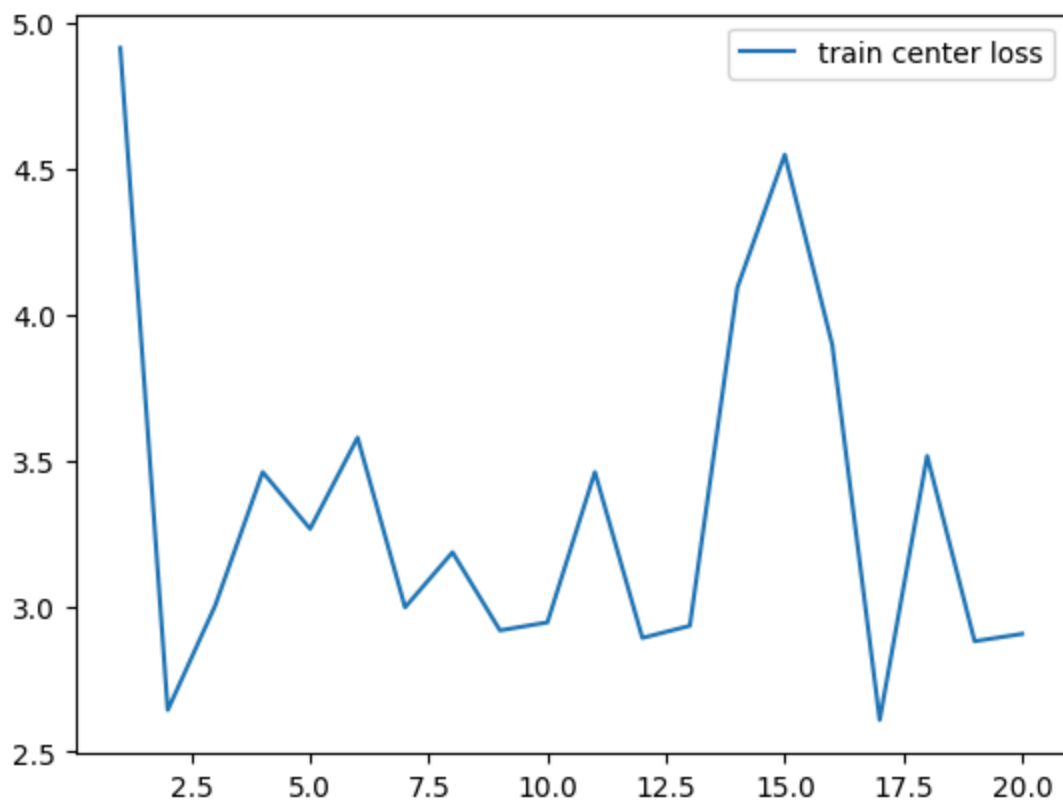
```
In [29]: import matplotlib.pyplot as plt
plt.plot(np.arange(1, len(train_losses)+1), train_losses, label='train loss')
plt.plot(np.arange(1, len(val_losses)+1), val_losses, label='val loss')
plt.legend()
plt.show()
```



```
In [30]: import matplotlib.pyplot as plt
plt.plot(np.arange(1, len(train_losses)+1), train_triplet_losses, label='train triplet loss')
plt.plot(np.arange(1, len(val_losses)+1), val_triplet_losses, label='val triplet loss')
plt.legend()
plt.show()
```



```
In [31]: import matplotlib.pyplot as plt
plt.plot(np.arange(1, len(train_losses)+1), train_center_losses, label='train center loss')
plt.legend()
plt.show()
```



2.9 (TODO) Plot t-SNE

This section is similar to section 1.8 but the network is instead optimized using a combined triplet, center, and cross-entropy loss.

Instructions

TODO 12: Extract the feature vectors of the test set and store them as

`embeddings` : torch.FloatTensor = feature vectors of all images in the test set

`identities` : list or torch.Tensor or np.array = identities of all images in the test set

Hint

Use `FaceDataset` that is imported at `Common Dataset` section

WARNING!! Don't forget load its best weights and change to eval mode first

```
In [32]: test_batch_size = 32

# TODO 12: Extract the feature vectors of the test set and store them as
# `embeddings`: torch.FloatTensor = feature vectors of all images in the test set
# `identities`: list or torch.Tensor or np.array = identities of all images in the
# Hint      => Use `FaceDataset` that is imported at `Common Dataset` section
# WARNING!! => Don't forget load its best weights and change to eval mode first

face_dataset = FaceDataset(root_dir='large_prepared_data/test', transform=val_trans
face_loader = DataLoader(face_dataset, batch_size=test_batch_size, shuffle=False)

part2_model = TripletNetwork(num_classes)
part2_model.load_state_dict(torch.load('weights/best_triplet_weights.pth'))
part2_model.to(device)
part2_model.eval()

embeddings = []
identities = []

with torch.no_grad():
    for transformed_img, identity, img_narray in tqdm(face_loader):
        transformed_img, identity, img_narray = transformed_img.to(device), identity.to(device), img_narray.to(device)
        identities.extend(identity)

        features = part2_model.extract_feature(transformed_img)
        embeddings.append(features)

embeddings = torch.cat(embeddings).cpu().numpy()
```

```
c:\Users\Faith\miniconda3\envs\pattern\lib\site-packages\torchvision\models\_utils.p
y:208: UserWarning:
```

The parameter 'pretrained' is deprecated since 0.13 and may be removed in the future, please use 'weights' instead.

```
c:\Users\Faith\miniconda3\envs\pattern\lib\site-packages\torchvision\models\_utils.p
y:223: UserWarning:
```

Arguments other than a weight enum or `None` for 'weights' are deprecated since 0.13 and may be removed in the future. The current behavior is equivalent to passing `weights=None`.

```
0%|          | 0/10 [00:00<?, ?it/s]
```

```
In [62]: import time
        from sklearn.manifold import TSNE

        time_start = time.time()
        tsne = TSNE(n_components=2, verbose=1, perplexity=30, max_iter=3000)
        tsne_result = tsne.fit_transform(embeddings)
        print('t-SNE done! Time elapsed: {} seconds'.format(time.time()-time_start))
```

```
[t-SNE] Computing 91 nearest neighbors...
[t-SNE] Indexed 301 samples in 0.001s...
[t-SNE] Computed neighbors for 301 samples in 0.013s...
[t-SNE] Computed conditional probabilities for sample 301 / 301
[t-SNE] Mean sigma: 1.148626
[t-SNE] KL divergence after 250 iterations with early exaggeration: 51.946396
[t-SNE] KL divergence after 1550 iterations: 0.504198
t-SNE done! Time elapsed: 0.5986239910125732 seconds
```

```
In [63]: import plotly.express as px

        label = []
        idx = 0
        id2label = dict()
        for identity in identities:
            identity = int(identity)
            if identity not in id2label:
                id2label[identity] = idx
                idx += 1
            label.append(id2label[identity])

        df_subset = pd.DataFrame({'label': label})

        df_subset['tsne-2d-one'] = tsne_result[:,0]
        df_subset['tsne-2d-two'] = tsne_result[:,1]

        fig = px.scatter(df_subset, x="tsne-2d-one", y="tsne-2d-two", color="label", height=
fig.update_layout(margin=dict(l=0, r=0, b=0, t=0))
fig.show()
```

```
In [64]: import time
        from sklearn.manifold import TSNE
```



```
time_start = time.time()
tsne = TSNE(n_components=3, verbose=1, perplexity=30, max_iter=3000)
tsne_result = tsne.fit_transform(embeddings)
print('t-SNE done! Time elapsed: {} seconds'.format(time.time()-time_start))
```

```
[t-SNE] Computing 91 nearest neighbors...
[t-SNE] Indexed 301 samples in 0.000s...
[t-SNE] Computed neighbors for 301 samples in 0.014s...
[t-SNE] Computed conditional probabilities for sample 301 / 301
[t-SNE] Mean sigma: 1.148626
[t-SNE] KL divergence after 250 iterations with early exaggeration: 51.954124
[t-SNE] KL divergence after 1300 iterations: 0.381682
t-SNE done! Time elapsed: 0.8962607383728027 seconds
```

```
In [65]: import plotly.express as px

label = []
idx = 0
id2label = dict()
for identity in identities:
    identity = int(identity)
    if identity not in id2label:
        id2label[identity] = idx
        idx += 1
    label.append(id2label[identity])

df_subset = pd.DataFrame({'label': label})

df_subset['tsne-3d-one'] = tsne_result[:,0]
df_subset['tsne-3d-two'] = tsne_result[:,1]
df_subset['tsne-3d-three'] = tsne_result[:,2]

fig = px.scatter_3d(df_subset, x="tsne-3d-one", y="tsne-3d-two", z="tsne-3d-three",
fig.update_layout(margin=dict(l=0, r=0, b=0, t=0))
fig.show()
```

2.10 (TODO) Embedding comparison

TODO 13: Compare the visualization of the triplet network to the siamese network. Which one is better and why?

Answer here: **I think it is better than the old one because it looks more group due to the loss we added.**

(Optional) Try other tricks to get higher quality of face embeddings such as

1. GeM Pooling (ref: <https://amaarora.github.io/2020/08/30/gempool.html>)
2. Arcface (ref: <https://arxiv.org/abs/1801.07698>)
3. Hard negative mining (ref: <https://omoindrot.github.io/triplet-loss>)
4. Semi-hard mining

and plot t-SNE to compare with vanilla triplet loss

Part 3 : Face recognition using InfoNCE loss

In contrast to the contrastive and triplet loss that only pulls an image pair of the same identity to come closer and push a different one away, some alternative approaches, such as the N-pair (ref: [paper](#)) and InfoNCE (ref: [paper](#)) loss, take multiple positive images or/and multiple negative images into consideration. In this part, you are going to implement a variant of the InfoNCE loss where multiple positive and negative images are simultaneously utilized during the learning process. Moreover, the loss function implemented in this homework also adopts temperature scaling, a concept widely applied in many state-of-the-art self-supervised learning such as SimCLR (ref: [paper](#)), instead of the margin used in triplet and vanilla contrastive loss.

3.1 (TODO) Dataset and DataLoader

In this section, you are going to implement a dataloader for the combined loss. The dataloader should return the tuple (anchor_img, pos_imgs, neg_imgs, anchor_label, pos_labels, neg_labels) .

- The positive image must have the same identity as the anchor image.
- The negative image must have a different identity from the anchor image.

this dataloader will sample both multiple positive images and multiple negative images per anchor image

```
In [37]: import copy

class InfoNCEDataset(Dataset):
    def __init__(self, root_dir, transform=None, num_pos=4, num_neg=8):
        np.random.seed(123)
        random.seed(123)
        self.root_dir = root_dir
        self.transform = transform
        self.num_pos = num_pos
        self.num_neg = num_neg

        label_df = pd.read_csv(f'{root_dir}/label_df.csv')
        label_df = label_df.groupby('identity')['filename'].apply(list).reset_index()
        self.images = []
        self.labels = []
        self.label2indices = dict()
        self.load_images_to_memory_(label_df)

    def load_images_to_memory_(self, label_df):
        # Load images and labels into memory
```

```

# We have to relabel from identities to 0,1,2,...,num_classes-1
# Relevant variables
# 1. `self.images` = PIL images (Also convert it to RGB)
# 2. `self.label2indices` = the dictionary storing a label as a key and the i
# 3. `self.labels` = labels of images (relabelled)
# Note: the index of images is from iterating over label_df
self.images = []
self.label2indices = defaultdict(list)
self.labels = []
img_idx = 0
label_idx = 0
identity2label = dict()
for idx in tqdm(range(len(label_df))):
    row = label_df.iloc[idx]
    identity, filenames = row['identity'], row['filenames']

    if identity not in identity2label:
        identity2label[identity] = label_idx
        label_idx += 1

    label = identity2label[identity]

    for filename in filenames:
        self.images.append(Image.open(f'{self.root_dir}/{filename}').convert('RGB'))
        self.labels.append(label)
        self.label2indices[label].append(img_idx)
        img_idx += 1

def __len__(self):
    return len(self.images)

def __getitem__(self, idx):
    """
    sample images labels
    [args]
    - idx is an index of an image

    [intermediate]
    - anchor image is the image according to idx
    - positive images (dim: [num_pos, num_feat]) is a list of image that has the
    - negative images (dim: [num_neg, num_feat]) is a list of image that has the

    [return]
    - transformed_anchor_img is the transformed anchor image
    - transformed_pos_imgs are the tensor of transformed positive images
    - transformed_neg_img are the tensor of transformed negative images
    - anchor_label is the label of the anchor image
    - pos_labels are the list of positive labels
    - neg_labels are the list of negative labels
    """
    anchor_img = self.images[idx]
    anchor_label = self.labels[idx]

    # TODO 14: sample positive images corresponding with the identity of anchor i

```

```

    ## condition:
    ## - there is no duplicated positive images
    ### Hint: Please use self.num_pos to determine the number of sampled positive
    pos_imgs_list = []
    pos_labels_list = []

    indices = self.label2indices[anchor_label]
    while (len(pos_imgs_list) < self.num_pos): # didn't cover the case where neg
        random_idx = np.random.choice(indices)
        if ((random_idx != idx) and (self.images[random_idx] not in pos_imgs_list)):
            pos_imgs_list.append(self.images[random_idx])
            pos_labels_list.append(self.labels[random_idx])

    # TODO 15: sample negative images that their identities differs from the iden
    ## condition:
    ## - the list of negative images can contain identical identities
    ## - there is no duplicated negative images
    ### Hint: Please use self.num_neg to determine the number of sampled negative
    neg_imgs_list = []
    neg_labels_list = []
    while (len(neg_imgs_list) < self.num_neg): # didn't cover the case where neg
        random_idx = np.random.randint(0, len(self.images))
        random_label = self.labels[random_idx]
        random_images = self.images[random_idx]

        while ((random_label == anchor_label) or (random_images in neg_imgs_list)):
            random_idx = np.random.randint(0, len(self.images))
            random_label = self.labels[random_idx]
            random_images = self.images[random_idx]

        neg_imgs_list.append(random_images)
        neg_labels_list.append(random_label)

    # TODO 16: utilize `self.transform` to convert anchor image, positive images,
    # WARNING!: Don't forget to convert to be tensors
    if (self.transform): # transform already have .ToTensor()
        anchor_img = self.transform(anchor_img)
        pos_imgs = torch.stack([self.transform(img) for img in pos_imgs_list])
        neg_imgs = torch.stack([self.transform(img) for img in neg_imgs_list])

    pos_labels = torch.tensor(pos_labels_list)
    neg_labels = torch.tensor(neg_labels_list)

    return anchor_img, pos_imgs, neg_imgs, anchor_label, pos_labels, neg_labels

```

```

In [38]: image_size = 224
         train_transform = transforms.Compose([
             transforms.Resize((256, 256)),
             transforms.CenterCrop(image_size),
             transforms.RandomHorizontalFlip(p=0.5),
             transforms.ToTensor(),
             transforms.Normalize(mean=[0.5319, 0.4399, 0.3929],
                                 std=[0.3076, 0.2898, 0.2907])
         ])

```

```

val_transform = transforms.Compose([
    transforms.Resize((256, 256)),
    transforms.CenterCrop(image_size),
    transforms.ToTensor(),
    transforms.Normalize(mean=[0.5319, 0.4399, 0.3929],
                        std=[0.3076, 0.2898, 0.2907])
])

train_batch_size = 16
val_batch_size = 16
test_batch_size = 16

# TODO 17: declare the datasets and the dataloaders
train_inforce_dataset = InfoNCEDataset(root_dir="large_prepared_data/train", transform=
train_inforce_dataloader = DataLoader(train_inforce_dataset, batch_size=train_batch_size)

val_inforce_dataset = InfoNCEDataset(root_dir="large_prepared_data/val", transform=
val_inforce_dataloader = DataLoader(val_inforce_dataset, batch_size=val_batch_size,

test_inforce_dataset = InfoNCEDataset(root_dir="large_prepared_data/test", transform=
test_inforce_dataloader = DataLoader(test_inforce_dataset, batch_size=test_batch_size)

0%|          | 0/80 [00:00<?, ?it/s]
0%|          | 0/10 [00:00<?, ?it/s]
0%|          | 0/10 [00:00<?, ?it/s]

```

3.2 (TODO) Image encoder

Image encoder consists of 3 modules

1. Feature extractor (ResNet18) is for mapping an image to a feature map
2. Global pooling (Global average pooling or use flatten instead) is for converting a feature map to a feature vector
3. Projection layer consists of 2 MLP layers to reduce dimension of feature vector together with a relu activation function at the middle of MLP layers

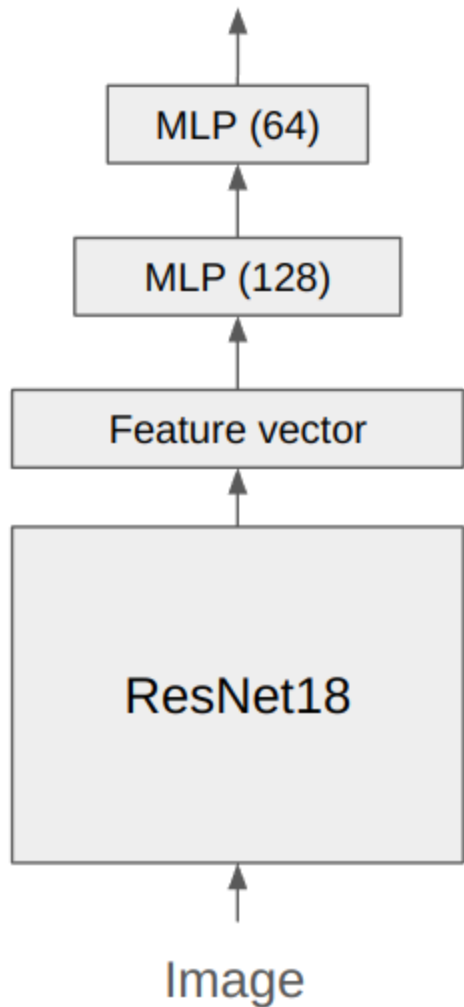
Note

1. ResNet18 can call via `torchvision` library (How to use torchvision: <https://pytorch.org/vision/stable/models.html>)
2. ResNet18 we use will not load pretrained weights
3. We will use ResNet18 only extracting feature maps.

Hint!

1. For global average pooling, read the documentation of `nn.AdaptiveAvgPool2d`

Figure 8 Image Encoder



In [39]: `from torchvision.models import resnet18`

```

class ImageEncoder(nn.Module):
    # TODO 18: implement the image encoder according to the above figure
    def __init__(self):
        super(ImageEncoder, self).__init__()
        self.resnet = resnet18()
        # print(self.resnet) # gets 512-dimensions feature
        self.resnet.fc = nn.Identity()
        self.fc = nn.Sequential(
            nn.Linear(512, 128),
            nn.ReLU(),
            nn.Linear(128, 64),
        )

    def extract_feature(self, x):
        features = self.resnet(x)
        flatten_features = features.flatten(start_dim=1)
        output = self.fc(flatten_features)
        return output

    def forward(self, anchor, pos, neg):
        # (batch, c, h, w)
        anchor_out = self.extract_feature(anchor)
  
```



```

batch_size, num_pos, c, h, w = pos.shape
pos_resaped = pos.reshape(-1, c, h, w)
pos_out = self.extract_feature(pos_resaped)
pos_out = pos_out.reshape(batch_size, num_pos, -1)

batch_size, num_neg, c, h, w = neg.shape
neg_resaped = neg.reshape(-1, c, h, w)
neg_out = self.extract_feature(neg_resaped)
neg_out = neg_out.reshape(batch_size, num_neg, -1)
return anchor_out, pos_out, neg_out

```

3.3 (TODO) InfoNCE loss

Information Noise Contrastive Estimation (InfoNCE) loss (ref: [blog](#), [paper](#)) is a self-supervised training objective aiming to maximize the agreement between positive samples and minimize the agreement between negative samples in the learned representation space. Although primarily designed for self-supervised learning, it could also be used in supervised setting by utilizing information from the labeled data.

When under self-supervision, images are sampled in batches, each of which is then augmented using two different augmentation policies. The contrastive loss is subsequently computed to maximize the similarity of the images of the same origin before augmentation while minimizing the similarity of the ones with different origins. In supervised learning, the objective maximizes the similarity of the images with the same identity (class) instead of origin. **In this part, we will train the InfoNCE loss under the supervised setting.**

To put it simply, the InfoNCE loss revolves around three types of images:

1. Anchor image: an initial image
2. Positive images: the sampled images with the same identity as the anchor image
3. Negative images: the sampled images whose identity differs from that of the anchor image

The objective is to maximize the cosine similarity between groups 1 and 2 while pushing groups 1 and 3 away. Thus, the InfoNCE maximizes the following objective:

$$InfoNCE\ Loss_i = \frac{1}{B \cdot P} \sum_{i=1}^B \sum_{p \in P(i)} -\log \left(\frac{e^{(\cos(f_i, f_p)/\tau)}}{e^{(\cos(f_i, f_p)/\tau)} + \sum_{n \in N(i)} e^{(\cos(f_i, f_n)/\tau)}} \right) \quad (1)$$

Notation

- B = batch size
- P = the number of positive images per anchor
- i = index of the anchor feature vector
- $P(i)$ = the sampled positive feature vectors
- $N(i)$ = the sampled negative feature vectors
- $\cos(f_i, f_p)$ = cosine similarity between f_i and f_p

- f_i = anchor feature vector
- f_p = positive feature vector
- f_n = negative feature vector
- τ = temperature (a scaling factor for increasing/reducing the magnitude of logits)

Hint use `nn.CrossEntropy` to compute the exponential part

```
In [40]: class InfoNCELoss(nn.Module):
# TODO 19: implement the InfoNCE loss
def __init__(self, device, temperature):
    super(InfoNCELoss, self).__init__()
    self.device = device
    self.temperature = temperature

def forward(self, anchor_feat, pos_feats, neg_feats):
    anchor_feat = anchor_feat.unsqueeze(1)
    anchor_feat = F.normalize(anchor_feat, p=2, dim=-1) # normalize in the last dim
    pos_feats = F.normalize(pos_feats, p=2, dim=-1)
    neg_feats = F.normalize(neg_feats, p=2, dim=-1)

    pos_similarity = torch.bmm(anchor_feat, torch.transpose(pos_feats, 1, 2)) # (batch, pos, pos)
    neg_similarity = torch.bmm(anchor_feat, torch.transpose(neg_feats, 1, 2)) # (batch, pos, neg)

    pos_sim = pos_similarity.transpose(1, 2)
    neg_sim = neg_similarity.expand(-1, pos_sim.shape[1], -1)

    logits = torch.cat([pos_sim, neg_sim], dim=2) # (batch, pos, neg+1)
    logits = logits.reshape(-1, logits.shape[2]) / self.temperature # (batch * pos, neg+1)
    labels = torch.zeros(logits.shape[0], dtype=torch.long, device=self.device)
    return F.cross_entropy(logits, labels) # default reduction is mean
```

3.4 (TODO) Initializing model, criterions, optimizers, hyparameters and scheduler

```
In [41]: temperature = 0.2

infonce_model = ImageEncoder()
device = 'cuda' if torch.cuda.is_available() else 'cpu'
train_infonce_criterion = InfoNCELoss(device, temperature)
val_infonce_criterion = InfoNCELoss(device, temperature)

infonce_optimizer = optim.Adam(infonce_model.parameters(), lr=5e-4)
infonce_scheduler = optim.lr_scheduler.ReduceLROnPlateau(infonce_optimizer, 'min',
```

3.5 (TODO) Training loop

Figure 9 How to connect the InfoNCE loss

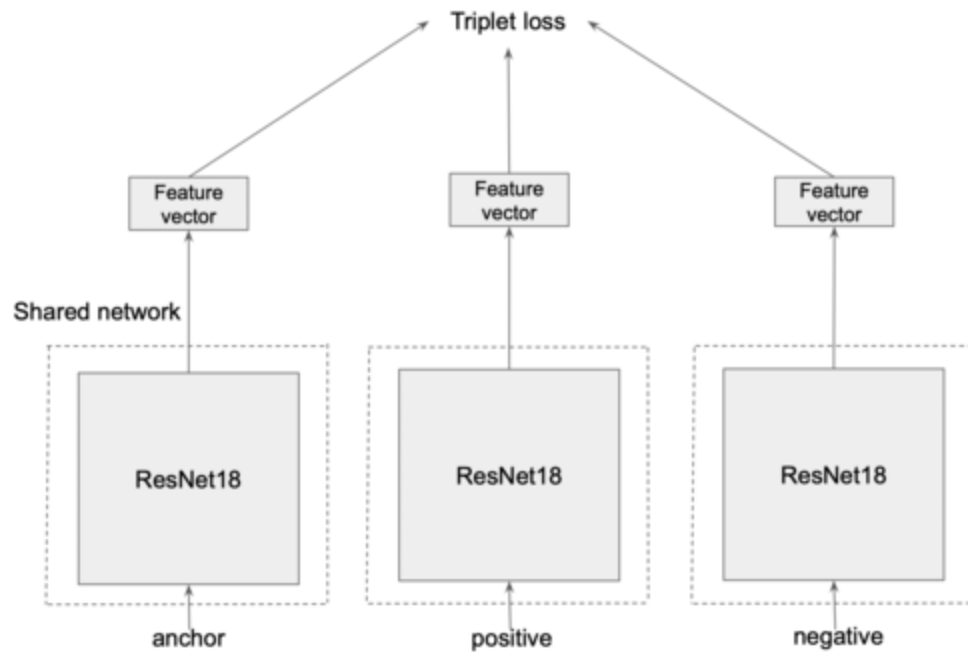


Fig 9 demonstrates how to connect multiple feature vectors to compute InfoNCE loss. The feature extractor shares all parameters between input images.

The training procedure of a infonce network consists of the following steps:

1. Foward pass `anchor_img`
2. Foward pass `pos_imgs`
3. Foward pass `neg_img`
4. Calculate all pairs cosine similarities between `anchor_img` and `pos_imgs` .
5. Calculate all pairs cosine similarities between `anchor_img` and `neg_imgs` .
6. Compute InfoNCE loss and take an average over them
7. Use the loss in step 6 to update the model
8. Repeat step 1-8 till converged

TODO 20: move training data to GPU memory, extract anchor, positive, negative features from images, and then use them to compute InfoNCE loss

TODO 21: set zero gradients

TODO 22 : back propagate at infonce network and step the optimizer

TODO 23: move validation data to GPU memory, extract anchor, positive, negative features from images, and then use them to compute InfoNCE loss

```
In [42]: num_epochs = 20
device = "cuda" if torch.cuda.is_available() else "cpu"
device
```

```
Out[42]: 'cuda'
```

```

In [43]: os.makedirs('weights', exist_ok=True)
best_weights_path = 'weights/best_infonce_weights.pth'
infonce_model.to(device)
min_val_loss = float('inf')

train_losses = []
val_losses = []

for epoch in tqdm(range(num_epochs)):
    infonce_model.train()
    total_train_loss = 0
    for anchor_img, pos_imgs, neg_imgs, _, _, _ in tqdm(train_infonce_dataloader):
        # TODO 20: feed data to the infonce model and compute infonce loss
        anchor_img, pos_imgs, neg_imgs = anchor_img.to(device), pos_imgs.to(device)
        anchor_feat, pos_feats, neg_feats = infonce_model(anchor_img, pos_imgs, neg_imgs)
        train_loss = train_infonce_criterion(anchor_feat, pos_feats, neg_feats)

        total_train_loss += train_loss.item()

        # TODO 21: set zero gradients
        infonce_optimizer.zero_grad()

        # TODO 22 : back propagate at infonce network and step the optimizer
        train_loss.backward()
        infonce_optimizer.step()

    current_train_loss = total_train_loss / len(train_infonce_dataloader)
    train_losses.append(current_train_loss)

    total_val_loss = 0
    infonce_model.eval()

    for val_anchor_img, val_pos_imgs, val_neg_imgs, _, _, _ in val_infonce_dataloader:
        # TODO 23: feed data to the infonce model and compute infonce loss
        val_anchor_img, val_pos_imgs, val_neg_imgs = val_anchor_img.to(device), val_pos_imgs.to(device), val_neg_imgs.to(device)
        with torch.no_grad():
            anchor_feat, pos_feats, neg_feats = infonce_model(val_anchor_img, val_pos_imgs, val_neg_imgs)
            val_loss = val_infonce_criterion(anchor_feat, pos_feats, neg_feats)

        total_val_loss += val_loss.item()

    current_val_loss = total_val_loss / len(val_infonce_dataloader)
    val_losses.append(current_val_loss)

    if current_val_loss < min_val_loss:
        min_val_loss = current_val_loss
        torch.save(infonce_model.state_dict(), best_weights_path)
    print(f'Epoch {epoch+1} '
          f'- Train loss = {current_train_loss:.4f} '
          f'- Val loss = {current_val_loss:.4f} '
          f'- best min_val_loss = {min_val_loss:.4f} '
          f'- lr = {infonce_optimizer.param_groups[0]["lr"]:.8f}')
    infonce_scheduler.step(current_val_loss)

```

```

0%|          | 0/20 [00:00<?, ?it/s]
0%|          | 0/153 [00:00<?, ?it/s]

```

```
Epoch 1 - Train loss = 2.0169 - Val loss = 1.9313 - best min_val_loss = 1.9313 - lr
= 0.00050000
0%|          | 0/153 [00:00<?, ?it/s]
Epoch 2 - Train loss = 1.8743 - Val loss = 1.6814 - best min_val_loss = 1.6814 - lr
= 0.00050000
0%|          | 0/153 [00:00<?, ?it/s]
Epoch 3 - Train loss = 1.7055 - Val loss = 1.5039 - best min_val_loss = 1.5039 - lr
= 0.00050000
0%|          | 0/153 [00:00<?, ?it/s]
Epoch 4 - Train loss = 1.5758 - Val loss = 1.4032 - best min_val_loss = 1.4032 - lr
= 0.00050000
0%|          | 0/153 [00:00<?, ?it/s]
Epoch 5 - Train loss = 1.4836 - Val loss = 1.4636 - best min_val_loss = 1.4032 - lr
= 0.00050000
0%|          | 0/153 [00:00<?, ?it/s]
Epoch 6 - Train loss = 1.3658 - Val loss = 1.4670 - best min_val_loss = 1.4032 - lr
= 0.00050000
0%|          | 0/153 [00:00<?, ?it/s]
Epoch 7 - Train loss = 1.3109 - Val loss = 1.4070 - best min_val_loss = 1.4032 - lr
= 0.00050000
0%|          | 0/153 [00:00<?, ?it/s]
Epoch 8 - Train loss = 1.1795 - Val loss = 1.2644 - best min_val_loss = 1.2644 - lr
= 0.00050000
0%|          | 0/153 [00:00<?, ?it/s]
Epoch 9 - Train loss = 1.1830 - Val loss = 1.4409 - best min_val_loss = 1.2644 - lr
= 0.00050000
0%|          | 0/153 [00:00<?, ?it/s]
Epoch 10 - Train loss = 1.0382 - Val loss = 1.3693 - best min_val_loss = 1.2644 - lr
= 0.00050000
0%|          | 0/153 [00:00<?, ?it/s]
Epoch 11 - Train loss = 0.9844 - Val loss = 1.2842 - best min_val_loss = 1.2644 - lr
= 0.00050000
0%|          | 0/153 [00:00<?, ?it/s]
Epoch 12 - Train loss = 0.8501 - Val loss = 1.3080 - best min_val_loss = 1.2644 - lr
= 0.00050000
0%|          | 0/153 [00:00<?, ?it/s]
Epoch 13 - Train loss = 0.8204 - Val loss = 1.1710 - best min_val_loss = 1.1710 - lr
= 0.00050000
0%|          | 0/153 [00:00<?, ?it/s]
Epoch 14 - Train loss = 0.8251 - Val loss = 1.2929 - best min_val_loss = 1.1710 - lr
= 0.00050000
0%|          | 0/153 [00:00<?, ?it/s]
Epoch 15 - Train loss = 0.7266 - Val loss = 1.1642 - best min_val_loss = 1.1642 - lr
= 0.00050000
0%|          | 0/153 [00:00<?, ?it/s]
Epoch 16 - Train loss = 0.6624 - Val loss = 1.2796 - best min_val_loss = 1.1642 - lr
= 0.00050000
0%|          | 0/153 [00:00<?, ?it/s]
Epoch 17 - Train loss = 0.5567 - Val loss = 1.3511 - best min_val_loss = 1.1642 - lr
= 0.00050000
0%|          | 0/153 [00:00<?, ?it/s]
Epoch 18 - Train loss = 0.6583 - Val loss = 1.3832 - best min_val_loss = 1.1642 - lr
= 0.00050000
0%|          | 0/153 [00:00<?, ?it/s]
Epoch 19 - Train loss = 0.5802 - Val loss = 1.4736 - best min_val_loss = 1.1642 - lr
= 0.00050000
```

```
0%|          | 0/153 [00:00<?, ?it/s]
Epoch 20 - Train loss = 0.6101 - Val loss = 1.2427 - best min_val_loss = 1.1642 - lr
= 0.00050000
```

3.6 Visualization

The visualization below displays an anchor, positive, and negative image and their respective similarity.

```
In [44]: # Showing images
def imshow(img, text=None):
    npimg = img.numpy()
    plt.axis("off")
    if text:
        plt.text(20, 8, text, style='italic', fontweight='bold',
                 bbox={'facecolor': 'white', 'alpha': 0.8, 'pad': 10})

    plt.imshow(np.transpose(npimg, (1, 2, 0)))
    plt.show()

best_weights_path = 'weights/best_infonce_weights.pth'
infonce_model.to(device)
infonce_model.load_state_dict(torch.load(best_weights_path))
infonce_model.eval()

test_anchor_imgs, batch_test_pos_imgs, batch_test_neg_imgs, test_anchor_ids, _, _ =

test_pos_imgs = batch_test_pos_imgs[:, 0, :, :]
test_neg_imgs = batch_test_neg_imgs[:, 0, :, :]

test_anchor_imgs, test_pos_imgs, test_neg_imgs = test_anchor_imgs.to(device), test_
with torch.no_grad():
    test_anchor_feats = infonce_model.extract_feature(test_anchor_imgs)
    test_pos_feats = infonce_model.extract_feature(test_pos_imgs)
    test_neg_feats = infonce_model.extract_feature(test_neg_imgs)

class UnNormalize(object):
    def __init__(self, mean, std):
        self.mean = mean
        self.std = std

    def __call__(self, tensor):
        """
        Args:
            tensor (Tensor): Tensor image of size (C, H, W) to be normalized.
        Returns:
            Tensor: Normalized image.
        """
        for t, m, s in zip(tensor, self.mean, self.std):
            t.mul_(s).add_(m)
            # The normalize code -> t.sub_(m).div_(s)
        return tensor

unnormalizer = UnNormalize(mean=[0.5319, 0.4399, 0.3929],
```

```
std=[0.3076, 0.2898, 0.2907])
```

```
zip_test_data = zip(test_anchor_feats, test_pos_feats, test_neg_feats, test_anchor_
for test_anchor_feat, test_pos_feat, test_neg_feat, test_anchor_img, test_pos_img,
test_anchor_img = unnormalizer(test_anchor_img.detach().cpu().unsqueeze(0))
test_pos_img = unnormalizer(test_pos_img.detach().cpu().unsqueeze(0))
test_neg_img = unnormalizer(test_neg_img.detach().cpu().unsqueeze(0))
concatenated = torch.cat((test_anchor_img, test_pos_img, test_neg_img), 0)
anc_pos_similarity = F.cosine_similarity(test_anchor_feat.unsqueeze(0), test_po
anc_neg_similarity = F.cosine_similarity(test_anchor_feat.unsqueeze(0), test_ne
imshow(torchvision.utils.make_grid(concatenated), f'Anc-Pos similarity: {anc_po
```

Clipping input data to the valid range for imshow with RGB data ([0..1] for floats or [0..255] for integers). Got range [0.0..1.1659949].



Clipping input data to the valid range for imshow with RGB data ([0..1] for floats or [0..255] for integers). Got range [0.0..1.1742941].



Clipping input data to the valid range for imshow with RGB data ([0..1] for floats or [0..255] for integers). Got range [0.0..1.1742941].



Clipping input data to the valid range for imshow with RGB data ([0..1] for floats or [0..255] for integers). Got range [0.0..1.1659949].



Clipping input data to the valid range for imshow with RGB data ([0..1] for floats or [0..255] for integers). Got range [0.0..1.1120508].



Clipping input data to the valid range for imshow with RGB data ([0..1] for floats or [0..255] for integers). Got range [0.0..1.1742941].



Clipping input data to the valid range for imshow with RGB data ([0..1] for floats or [0..255] for integers). Got range [0.0..1.1742941].



Clipping input data to the valid range for imshow with RGB data ([0..1] for floats or [0..255] for integers). Got range [0.0..1.1327987].



Clipping input data to the valid range for imshow with RGB data ([0..1] for floats or [0..255] for integers). Got range [0.0..1.1452472].



Clipping input data to the valid range for imshow with RGB data ([0..1] for floats or [0..255] for integers). Got range [0.0..1.1410978].



Clipping input data to the valid range for imshow with RGB data ([0..1] for floats or [0..255] for integers). Got range [0.0..1.1327987].



Clipping input data to the valid range for imshow with RGB data ([0..1] for floats or [0..255] for integers). Got range [0.0..1.1742941].



Clipping input data to the valid range for imshow with RGB data ([0..1] for floats or [0..255] for integers). Got range [0.0..1.1742941].



Clipping input data to the valid range for imshow with RGB data ([0..1] for floats or [0..255] for integers). Got range [0.0..1.1742941].



Clipping input data to the valid range for imshow with RGB data ([0..1] for floats or [0..255] for integers). Got range [0.0..1.1742941].

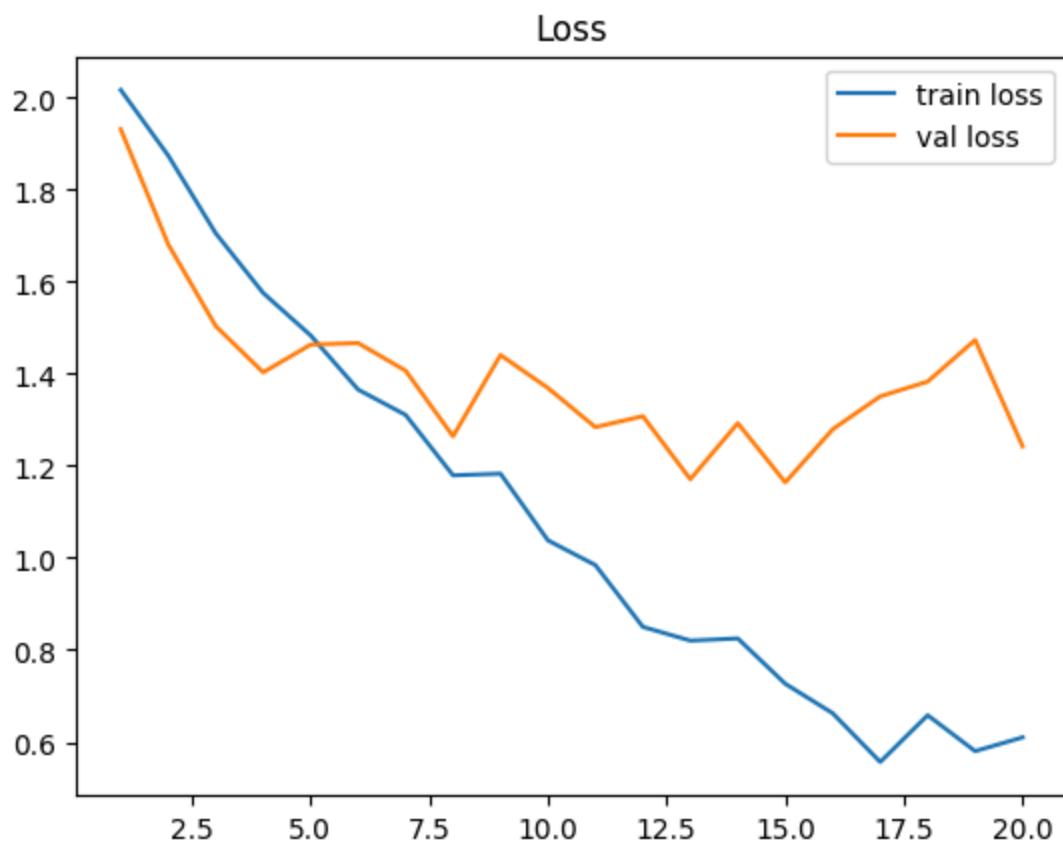


Clipping input data to the valid range for imshow with RGB data ([0..1] for floats or [0..255] for integers). Got range [0.0..1.1742941].



3.7 Plot loss history

```
In [45]: import matplotlib.pyplot as plt
plt.plot(np.arange(1, len(train_losses)+1), train_losses, label='train loss')
plt.plot(np.arange(1, len(val_losses)+1), val_losses, label='val loss')
plt.legend()
plt.title('Loss')
plt.show()
```



3.8 (TODO) Plot t-SNE

This section is similar to section 1.8

Instructions

TODO 24: Extract the feature vectors of the test set and store them as

`embeddings` : torch.FloatTensor = feature vectors of all images in the test set

`identities` : list or torch.Tensor or np.array = identities of all images in the test set

Hint

Use `FaceDataset` that is imported at `Common Dataset` section

WARNING!! Don't forget load its best weights and change to eval mode first

```
In [46]: test_batch_size = 32

# TODO 24: Extract the feature vectors of the test set and store them as
# `embeddings`: torch.FloatTensor = feature vectors of all images in the test set
# `identities`: list or torch.Tensor or np.array = identities of all images in the
# Hint      => Use `FaceDataset` that is imported at `Common Dataset` section
# WARNING!! => Don't forget load its best weights and change to eval mode first
face_dataset = FaceDataset(root_dir='large_prepared_data/test', transform=val_trans
face_loader = DataLoader(face_dataset, batch_size=test_batch_size, shuffle=False)

part3_model = ImageEncoder()
part3_model.load_state_dict(torch.load(best_weights_path))
part3_model.to(device)
part3_model.eval()

embeddings = []
identities = []

with torch.no_grad():
    for transformed_img, identity, img_narray in tqdm(face_loader):
        transformed_img = transformed_img.to(device)
        feature = part3_model.extract_feature(transformed_img)

        identities.extend(identity)
        embeddings.append(feature.cpu())

embeddings = torch.cat(embeddings).cpu()

0%|          | 0/10 [00:00<?, ?it/s]
```

```
In [72]: import time
from sklearn.manifold import TSNE

time_start = time.time()
tsne = TSNE(n_components=2, verbose=1, perplexity=30, max_iter=3000)
tsne_result = tsne.fit_transform(embeddings)
print('t-SNE done! Time elapsed: {} seconds'.format(time.time()-time_start))
```

```
[t-SNE] Computing 91 nearest neighbors...
[t-SNE] Indexed 301 samples in 0.000s...
[t-SNE] Computed neighbors for 301 samples in 0.014s...
[t-SNE] Computed conditional probabilities for sample 301 / 301
[t-SNE] Mean sigma: 1.148626
[t-SNE] KL divergence after 250 iterations with early exaggeration: 51.926952
[t-SNE] KL divergence after 1400 iterations: 0.564041
t-SNE done! Time elapsed: 0.5459458827972412 seconds
```

```
In [73]: import plotly.express as px

label = []
idx = 0
id2label = dict()
for identity in identities:
    identity = int(identity)
    if identity not in id2label:
        id2label[identity] = idx
        idx += 1
    label.append(id2label[identity])

df_subset = pd.DataFrame({'label': label})

df_subset['tsne-2d-one'] = tsne_result[:,0]
df_subset['tsne-2d-two'] = tsne_result[:,1]

fig = px.scatter(df_subset, x="tsne-2d-one", y="tsne-2d-two", color="label", height=
fig.update_layout(margin=dict(l=0, r=0, b=0, t=0))
fig.show()
```

```
In [74]: import time
from sklearn.manifold import TSNE

time_start = time.time()
tsne = TSNE(n_components=3, verbose=1, perplexity=30, max_iter=3000)
tsne_result = tsne.fit_transform(embeddings)
print('t-SNE done! Time elapsed: {} seconds'.format(time.time()-time_start))
```

```
[t-SNE] Computing 91 nearest neighbors...
[t-SNE] Indexed 301 samples in 0.001s...
[t-SNE] Computed neighbors for 301 samples in 0.014s...
[t-SNE] Computed conditional probabilities for sample 301 / 301
[t-SNE] Mean sigma: 1.148626
[t-SNE] KL divergence after 250 iterations with early exaggeration: 51.949131
[t-SNE] KL divergence after 950 iterations: 0.360757
t-SNE done! Time elapsed: 0.6691272258758545 seconds
```

```
In [75]: import plotly.express as px

label = []
idx = 0
id2label = dict()
for identity in identities:
    identity = int(identity)
    if identity not in id2label:
        id2label[identity] = idx
```

```

    idx += 1
    label.append(id2label[identity])

df_subset = pd.DataFrame({'label': label})

df_subset['tsne-3d-one'] = tsne_result[:,0]
df_subset['tsne-3d-two'] = tsne_result[:,1]
df_subset['tsne-3d-three'] = tsne_result[:,2]

fig = px.scatter_3d(df_subset, x="tsne-3d-one", y="tsne-3d-two", z="tsne-3d-three",
fig.update_layout(margin=dict(l=0, r=0, b=0, t=0))
fig.show()

```

3.9 (TODO) Please analyze and compare between the t-SNE of triplet network the t-SNE of InfoNCE network. Why are those t-SNEs not in the same way?

TODO 25: Answer the above question

Hint The reason is about our dataset

Answer: **I think both work wells. The Triplet Loss looks more clustered due to the loss which cares about the distance to centroids, but this one didn't so it doesn't preserve the distance to the center of the group with some outliers showing there. Maybe another cause might be the small batch size which makes the InfoNCE converges harder.**

Part 4 : Face verification evaluation

In HW3, we use the ROC curve to measure the performance of the face verification task. Similarly, we will use this metric to measure the performance of our NN.

TODO 26: Use the Siamese network, network trained using triplet loss, and network trained using InfoNCE loss to extract the image features from the test set.

```

In [51]: device = 'cuda' if torch.cuda.is_available() else 'cpu'

val_transform = transforms.Compose([
    transforms.Resize((256, 256)),
    transforms.CenterCrop(img_size),
    transforms.ToTensor(),
    transforms.Normalize(mean=[0.5319, 0.4399, 0.3929],
                          std=[0.3076, 0.2898, 0.2907])
])

test_siamese_dataset = SiameseDataset(root_dir='large_prepared_data/test', transfor

```



```

test_siamese_dataloader = DataLoader(test_siamese_dataset, batch_size=test_batch_si

siamese_embeddings1 = []
siamese_embeddings2 = []
triplet_embeddings1 = []
triplet_embeddings2 = []
infonce_embeddings1 = []
infonce_embeddings2 = []
labels = []

siamese_model.to(device)
triplet_model.to(device)
infonce_model.to(device)

siamese_model.load_state_dict(torch.load('weights/best_siamese_weights.pth'))
triplet_model.load_state_dict(torch.load('weights/best_triplet_weights.pth'))
infonce_model.load_state_dict(torch.load('weights/best_infonce_weights.pth'))

siamese_model.eval()
triplet_model.eval()
infonce_model.eval()

for batch_img1, batch_img2, batch_label in tqdm(test_siamese_dataloader):
    batch_img1, batch_img2, batch_label = batch_img1.to(device), batch_img2.to(device)
    with torch.no_grad():
        # TODO 26: extract features with both siamese network, triplet network, infonce
        # and keep embeddings in provided lists according to variable namesA
        siamese_embeddings1.append(siamese_model.extract_feature(batch_img1))
        siamese_embeddings2.append(siamese_model.extract_feature(batch_img2))
        triplet_embeddings1.append(triplet_model.extract_feature(batch_img1))
        triplet_embeddings2.append(triplet_model.extract_feature(batch_img2))
        infonce_embeddings1.append(infonce_model.extract_feature(batch_img1))
        infonce_embeddings2.append(infonce_model.extract_feature(batch_img2))

    labels.append(batch_label)

siamese_embeddings1 = torch.cat(siamese_embeddings1)
siamese_embeddings2 = torch.cat(siamese_embeddings2)
triplet_embeddings1 = torch.cat(triplet_embeddings1)
triplet_embeddings2 = torch.cat(triplet_embeddings2)
infonce_embeddings1 = torch.cat(infonce_embeddings1)
infonce_embeddings2 = torch.cat(infonce_embeddings2)
labels = torch.cat(labels)

```

```
0%|          | 0/10 [00:00<?, ?it/s]
```

TODO 27: Measure the similarity score between the two feature vectors with cosine similarity.

HINT You can use `nn.CosineSimilarity` (ref:

<https://pytorch.org/docs/stable/generated/torch.nn.CosineSimilarity.html>).

```
In [52]: def compute_pairs_cosine_sim(input1, input2):
# TODO 27: implement cosine similarity function that can compute multiple pairs a
cos = nn.CosineSimilarity(dim=1) # it is a module
return cos(input1, input2)

siamese_scores = compute_pairs_cosine_sim(siamese_embeddings1, siamese_embeddings2)
triplet_scores = compute_pairs_cosine_sim(triplet_embeddings1, triplet_embeddings2)
infonce_scores = compute_pairs_cosine_sim(infonce_embeddings1, infonce_embeddings2)
```

TODO 28

Plot a ROC curve to compare the performance siamese, triplet, infonce networks. Which one is better and why?

```
In [53]: import matplotlib.pyplot as plt
# TODO 28: calculate true positive rate and false positive rate
# and plot both the ROC curve of siamese network and the ROC curve of triplet network
# together with AUC score
from sklearn.metrics import roc_curve, auc

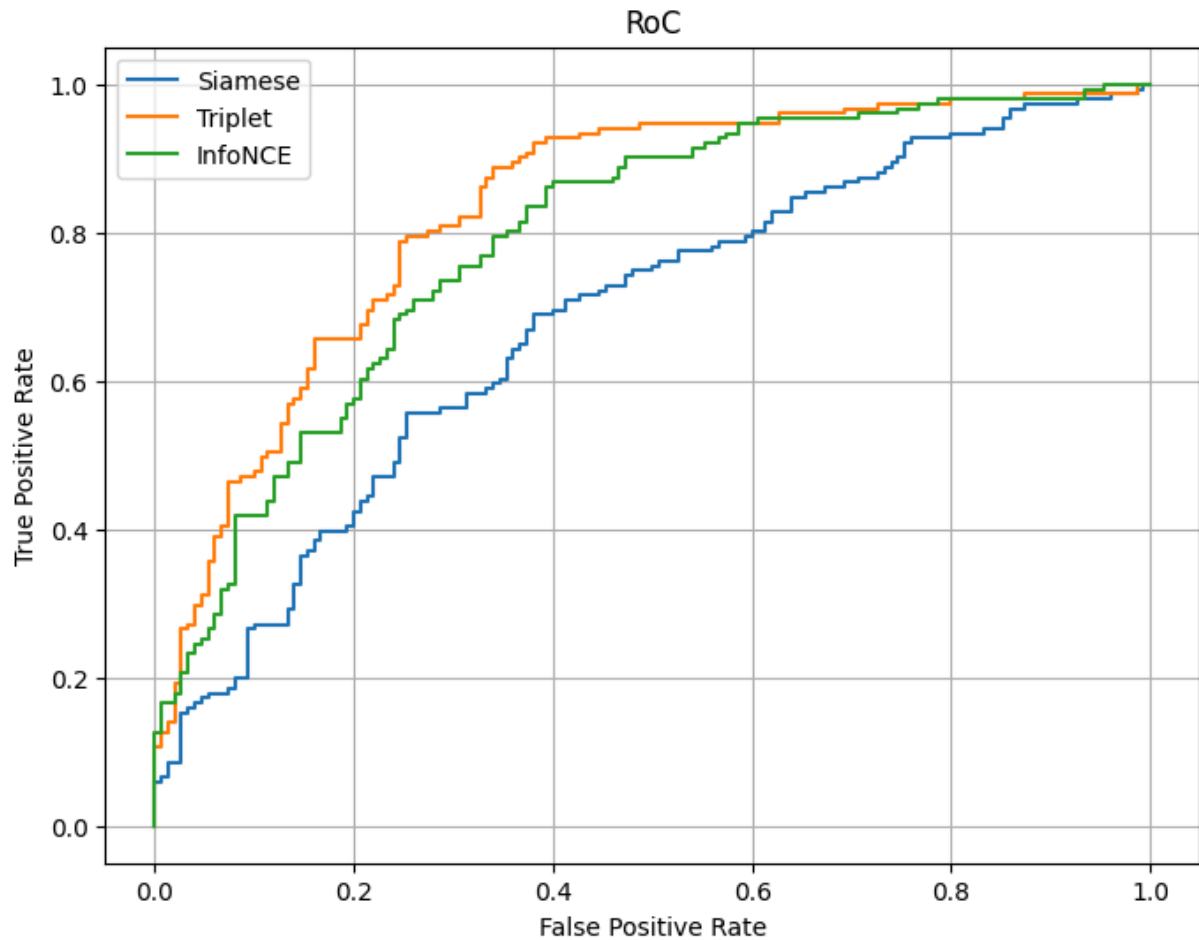
labels_np = labels.detach().cpu().numpy()

fpr_siamese, tpr_siamese, thresh_siamese = roc_curve(labels_np, siamese_scores.detach())
auc_siamese = auc(fpr_siamese, tpr_siamese) # (x, y)
fpr_triplet, tpr_triplet, thresh_triplet = roc_curve(labels_np, triplet_scores.detach())
auc_triplet = auc(fpr_triplet, tpr_triplet)
fpr_infonce, tpr_infonce, thresh_infonce = roc_curve(labels_np, infonce_scores.detach())
auc_infonce = auc(fpr_infonce, tpr_infonce)

plt.figure(figsize=(8, 6))
plt.plot(fpr_siamese, tpr_siamese, label="Siamese")
plt.plot(fpr_triplet, tpr_triplet, label="Triplet")
plt.plot(fpr_infonce, tpr_infonce, label="InfoNCE")

plt.xlabel("False Positive Rate")
plt.ylabel("True Positive Rate")
plt.title("RoC")
plt.legend()
plt.grid(True)
plt.show()

print(f"Siamese AUC: {auc_siamese:.3f}")
print(f"Triplet AUC: {auc_triplet:.3f}")
print(f"InfoNCE AUC: {auc_infonce:.3f}")
```



Siamese AUC: 0.680
Triplet AUC: 0.831
InfoNCE AUC: 0.792

Triplet performs the best.

Why? I think both Triplet and InfoNCE outperform Siamese because they encode relative distance relationship.

But, there are not much different in Triplet and InfoNCE.